

AA-11

Team Execution Handbook

One hundred and thirty numbered tasks with owners, steps and checkable outputs

30 August 2026 · Rev 1.0 — baseline · AA-11 of thirteen

NUMBERED TASKS 130	WEEKS W1 to W10	CHECKABLE OUTPUTS 260
CI GATES 12	MIGRATIONS 8	DONE CRITERIA 20

This is the document the four engineers work from every day. It is not persuasive and it does not argue for the programme; AA-10 does that. Every task here has an owner, an ordered list of steps and an acceptance line that is checkable by someone who did not write the code. Read section 11.1 once, keep sections 11.2 to 11.6 open in a tab, and work the week you are in. Where this handbook and a design document disagree, the design document wins and the task card is wrong.

11.1 HOW TO USE THIS HANDBOOK

130 tasks, 406 steps, 260 acceptance lines, and one rule about all of them

NUMBERED TASKS

130

across W1 to W10, none unowned

ORDERED STEPS

406

the suggested route, not a contract

ACCEPTANCE LINES

260

these are the contract

BUSIEST WEEK

W2 · 18

permissions, audit and the admin shell

LIGHTEST WEEK

W10 · 9

pilot, runbooks and cutover

PORTAL SCREENS

91

41 admin, 26 supplier, 24 fleet

A task identifier reads P<week>-<dev>-<nn>. P4-BE2-03 is the third task assigned to BE-2 in Week 4. The prefix is P for phase because the week numbers here are programme weeks and not calendar weeks; if the programme starts late, W4 is still W4. The dev segment is one of the five identifiers in AA-10 section 10.2 — BE-1, BE-2, FE-1, FE-2, MOB-1 — or ALL for a whole-team activity such as a demo rehearsal or a contract sweep, which the DEV column below prints in lower case. The sequence number is per dev per week and carries no priority; ordering inside a week comes from the prose above each table, not from the number.

The two right-hand columns are not the same kind of thing and confusing them is the most common way a task goes wrong. STEPS is a route that worked when this plan was written. If you find a better route, take it and say so at the Wednesday sync. ACCEPTANCE is the definition of done for that task. It is not a suggestion, it is not a stretch goal, and it is not satisfied by code that would pass it if a missing dependency existed. Every acceptance line was written so that a second engineer who has not read your diff can check it in under two minutes.

A task is finished when it is demonstrable on the deployed host, not when the pull request merges. That distinction is deliberate and it will cost you time in W1 and W2 before it starts saving you time. Merged-but-not-deployed work hides configuration errors, missing environment variables, migration ordering mistakes and CORS mistakes until the Friday demo, which is exactly when they are most expensive. The deployment targets are named in AA-09 section 9.2; treat a deploy as part of the task, not as an afterthought that someone else batches on Thursday.

Some of these cards are wrong. A hundred and thirty tasks written before a line of the programme was built will contain ordering mistakes, screen numbers that a later design revision moved, and estimates that were optimistic. The response to a wrong card is to raise it at the Wednesday blocker sync, where the whole team hears the change and AA-10 section 10.6 can be consulted if the week now needs a cut. The response is never to silently re-scope it in your own branch. A silently re-scoped task is invisible until the Friday gate, by which point the week has already been lost and nobody knows why.

DEV	ROLE	PRIMARY SURFACE	TASKS
BE-1	Backend — platform	backoffice, partners, catalog, identity, platform	35
BE-2	Backend — fulfilment	orders, fleet, money, advisor	33
FE-1	Front end — admin + kit	atavishaala-portal-kit, atavishaala-admin-portal	27
FE-2	Front end — partner + fleet	atavishaala-supplier-portal, atavishaala-fleet-portal	20
MOB-1	Mobile — unaffected	atavishaala-api /app namespace consumer	1
all	Whole-team activities	demos, sweeps, rehearsals, cutover	14

THE FRIDAY GATE IS BINARY

A week is passed or explicitly failed at the Friday 16:00 demo gate against real data on the deployed hosts. There is no partial pass. If your task is 90 per cent done on Friday it is not done, it moves into the following week, and it consumes capacity that was already allocated. AA-10 section 10.5 lists the four syncs and what each one is for.

STEPS ARE ADVICE, ACCEPTANCE IS LAW

Deviating from the steps needs no permission. Deviating from an acceptance line needs a decision at the Wednesday sync, because the acceptance lines are what the demo gate is checked against and what the definition of done in section 11.6 assumes has already happened.

11.2 CONVENTIONS

12 rules that stop four people writing four codebases

None of these rules is interesting on its own. They exist because five repositories written by four people in ten weeks diverge faster than anyone expects, and because the review cost of a divergence is paid every week after it lands. Read them once and then let the linter and the reviewer enforce them; arguing about them in a pull request is a waste of a Wednesday.

TOPIC	RULE
Branches	feat/<area>-<short>, fix/<area>-<short>, chore/<area>-<short>. No personal branches.
Commits	Conventional commits. The scope is the module or the portal, e.g. feat(fleet): offer wave sweep.
PRs	One task ID per PR, named in the title. A PR that touches two repos is two PRs with a cross-link.
Reviews	BE-1 reviews BE-2 and the reverse. FE-1 reviews FE-2 and the reverse. Kit changes need both FE reviews.
Migrations	One migration per PR. Never edit an applied migration. Every migration has a rollback note in the PR body.
Contracts	Zod schema first, then generate. Never hand-edit the generated OpenAPI or client.
Kit changes	Additive within a minor. A breaking kit change needs a major bump and a coordinated portal PR.
Feature flags	Any feature that spans a week boundary ships behind a flag defaulting to off.
Error codes	SCREAMING_SNAKE, defined once in packages/shared/errors.ts. A new code needs a line in AA-04.
Time	Store UTC. Compute business hours in Asia/Kolkata. Never trust a client clock for an SLA.
Money	Integer paise everywhere. Format for display only, at the very edge, in the kit.
Logging	Pino structured JSON. Never log a masked field, even masked. Log identifiers, not values.

Branch names carry an area, not a person. A branch called feat/fleet-offer-wave tells a reviewer which module is in play and which of the two backend engineers should be reading it; a branch called ayushman/wip tells them nothing and cannot be triaged when its author is on leave. The area segment should match the module or portal name used in AA-02, so that a branch, a commit scope and a task card all name the same thing. The short segment is two or three words, not a sentence, and never a ticket number, because the ticket number belongs in the pull request title where it is searchable.

The ban on an unexplained `// @ts-expect-error` is the one convention with no discretion attached. A suppression is a claim that the type system is wrong, and that claim needs somewhere to live longer than the diff that made it. Every suppression carries the URL of an issue that says what is actually wrong and what would remove it, and the typecheck gate in section 11.3 fails the build without one. The alternative that was rejected was a blanket ban on suppressions: it loses because two of our dependencies ship types that are genuinely wrong, and a blanket ban pushes engineers into casting through any, which is worse because it is invisible to grep.

Commit scope matters for the same reason branch area does. Conventional commits scoped to the module make the release notes for a week generate themselves, and they make `git log -- src/modules/money` readable when the ledger disagrees with a payout six weeks from now. One task identifier per pull request keeps this honest. When a task genuinely spans two repositories, it becomes two pull requests that reference each other, and the acceptance line is checked only once both are deployed.

11.3 THE CI GATES

12 checks on GitHub Actions, all free-tier minutes, most of them blocking

GATE	WHERE	RULE
Typecheck	all five repos	<code>tsc --noEmit</code> . Zero errors. No <code>// @ts-expect-error</code> without a linked issue.
Lint	all five repos	eslint with the eight custom rules above as errors, not warnings.
PII grep	all five repos	A pre-commit and CI grep for ten consecutive digits in a template literal or JSX text.
Secret scan	all five repos	gitleaks over the full history, not just the diff.
Range pin check	three portals	package.json must pin <code>@atavishaala/kit</code> and <code>/contract</code> to an exact version.

GATE	WHERE	RULE
Contract sweep	atavishaala-api	Every non-public route asserts 401 for the three wrong token audiences.
Permission sweep	atavishaala-api	Every admin route declares a permission that exists in the union.
Idempotency replay	atavishaala-api	Every money and offer fixture is replayed; the second response must be byte-identical.
FSM table test	atavishaala-api	All 28 shipment-state combinations drive reduceOrderStatus to the expected value.
Migration rehearsal	atavishaala-api	M1..M8 applied to a restored Neon branch, then a derived-status diff of zero rows.
CSP check	three portals	Playwright asserts the deployed CSP header has no unsafe-inline and no wildcard connect-src.
Bundle budget	three portals	Initial JS under 220 KB gzipped per portal. Fails the build above it.

Ten of the twelve gates are merge-blocking. Typecheck, lint, the PII grep, the secret scan, the range pin check, the contract sweep, the permission sweep, idempotency replay, the FSM table test and the CSP check all fail the pull request. The two advisory ones are the migration rehearsal and the bundle budget, and they are advisory for different reasons. The rehearsal needs a restored Neon branch, which is a shared resource on the free plan and cannot be created concurrently for four engineers, so it runs nightly against main and reports rather than blocking a feature branch. The bundle budget starts advisory in W1 and W2 while the kit is still growing, and becomes blocking at the end of W3 once the kit surface has stabilised; leaving it blocking from day one produces a week of pointless argument about whether a chart library counts.

The PII grep is deliberately implemented twice, as a pre-commit hook and again as a CI gate, and that duplication is not an oversight. The hook is there because the failure it catches is unrecoverable: once ten consecutive digits of a real phone number reach a public commit, rotating them is not possible the way rotating a leaked key is, and the git history keeps them forever. Catching it before the commit object exists is the only cheap fix. The CI copy exists because hooks are local, skippable with --no-verify and absent on a fresh clone, so the hook is a convenience and the CI gate is the actual control. Both use the same script from packages/shared so they cannot drift apart.

None of these gates is a substitute for review. The permission sweep proves that every admin route declares a permission that exists; it cannot prove the route declares the right one. The FSM table test proves all twenty-eight shipment-state combinations reduce to the expected order status; it cannot prove the expected values in the fixture are the ones the business wants. AA-08 section 8.4 and AA-03 section 3.6 are the authorities for those two questions respectively, and a reviewer who has not read them is not qualified to approve the diff.

11.4 MIGRATIONS

8 forward-only steps, M1 to M8, applied in that order and never edited

ID	AREA	TABLES AND ALTERS	BACKFILL	NOTE
M1	backoffice identity	staff_users, staff_roles, staff_role_grants, role_permissions, staff_totp	None. Purely additive.	Seeds the 9 system roles and the 88 permission rows. Creates the first SUPER_ADMIN from BOOTSTRAP_ADMIN_EMAIL, which the migration then requires to enrol TOTP on first login.
M2	partner tenancy	partner_orgs, partner_users, partner_kyc, partner_bank_accounts, partner_rate_cards	Adds nullable products.partner_org_id.	Backfills every existing product to the seeded house org ATV-HOUSE so the column can be made NOT NULL in M3 without downtime.
M3	catalog ownership	ALTER products.partner_org_id SET NOT NULL; new partial index on products (partner_org_id, moderation_status)	Requires M2 backfill complete.	Adds products.moderation_status DRAFT IN_REVIEW APPROVED REJECTED DELISTED with default APPROVED for existing rows only, then flips the column default to DRAFT.
M4	shipment split	shipments, shipment_items, order_events	THE risky one. Backfills one shipment per existing order.	Runs in a single transaction over a table that is small at MVP. The backfill assigns every existing order a single shipment against ATV-HOUSE, copies status via a mapping table, and writes synthetic order_events from the existing status timestamps.
M5	fleet tenancy	fleet_orgs, fleet_users, fleet_kyc, fleet_bank_accounts, fleet_service_areas, riders, vehicles	None. Additive.	Seeds one internal fleet org ATV-SELF at tier 1 so orders can still be delivered by staff while no third-party fleet has onboarded.

ID	AREA	TABLES AND ALTERS	BACKFILL	NOTE
M6	delivery jobs	delivery_jobs, job_offers, pod_records	Requires M4 (needs shipment_id) and M5 (needs fleet_org_id).	Creates one delivery_job per existing shipment in a terminal state, marked legacy_backfill = true, so historical orders render in the fleet timeline without offers.
M7	settlement	partner_payouts, payout_runs, partner_ledger, fleet_payouts, fleet_ledger, cod_remittances	Requires M2 and M5.	Installs the three append-only Postgres rules. Does NOT backfill historical payouts — the first run covers the first full week after go-live only, stated explicitly in the release note.
M8	governance	pii_disclosures, audit_log, idempotency_keys, saved_views, exports, notification_templates, webhook_deliveries	None. Additive.	Last because it is the only migration that can be applied at any time; it is deliberately kept out of the critical path so an audit-log schema change never blocks a money migration.

Migrations are forward-only. There is no down migration in this programme and writing one is not a favour to your colleagues. A down migration that drops a column is a loaded gun pointed at production data, and one that is never rehearsed is a lie in the repository; the honest position is that recovery from a bad migration is a restore from a Neon branch plus a replay, which is documented in AA-09 and has an owner. What every migration pull request does carry is a rollback note in the body: one paragraph stating what breaks if this migration is applied and the code that needs it is not deployed, and vice versa. That is the question that actually comes up at 22:00.

M4 is the only migration that touches existing rows in a way that can be wrong rather than merely absent. M2 adds a nullable column and fills it with a constant, and M3 tightens it; both are dull. M4 invents a shipment for every order that already exists, maps the old order status through a mapping table onto the new shipment state, and then writes synthetic order_events from whatever timestamps the old rows happen to carry. Every one of those three operations can produce a row that is structurally valid and semantically wrong, and because orders.status is derived by reduceOrderStatus() rather than stored, a wrong shipment state silently changes what a customer sees in the app tracker.

The rehearsal procedure is therefore not optional and it is not a nightly job that somebody glances at. Before M4 is applied anywhere that matters, BE-2 creates a Neon branch from the current production snapshot, applies M1 to M4 against that branch, and then runs a diff that computes reduceOrderStatus() over the new shipment rows and compares it to the status the old rows carried. The pass condition is zero differing rows, not a small number of explicable ones. The same procedure runs again in W9 for the full M1 to M8 chain, which is definition-of-done item D-18, and again as part of cutover in W10 with the branch discarded afterwards. A Neon branch is free and takes under a minute; there is no excuse in the budget for skipping it.

ONE MIGRATION PER PULL REQUEST, AND NEVER EDIT AN APPLIED MIGRATION

An applied migration is a historical fact about every database that has run it. Editing the file makes the repository disagree with reality on every environment where the old version already ran, and the disagreement surfaces months later as a rehearsal that passes on a fresh database and fails on a restored one. If M3 is wrong, M9 fixes it. This is the same reasoning that makes the ledgers in AA-03 section 3.8 append-only: a correction is a new row, never an UPDATE.

11.5 ENVIRONMENT

37 variables on the API, 6 on each portal, 43 to get wrong

Two kinds of variable live in this table and the distinction is the whole point of the section. A secret is a value whose disclosure is an incident: it is set in the hosting dashboard, it is never committed, it is never printed by a log line or an error page, and it is rotatable by the procedure in AA-09. A config value is a knob: it shapes behaviour, it belongs in review because changing it changes what the product does, and several of the config rows below are load-bearing policy rather than tuning. MANIFEST_WINDOW_MINUTES, POD_GEOFENCE_METRES and COD_VARIANCE_BLOCK_BPS are the clearest examples; each one is a control described in AA-08 that happens to be expressed as a number in an environment variable.

NAME	KIND	NOTE
DATABASE_URL	secret	Neon pooled connection string with sslmode=require.
DIRECT_URL	secret	Neon direct connection, used by prisma migrate only.
JWT_PRIVATE_KEY	secret	RS256 private key, PEM, base64-encoded to survive the env.
JWT_PUBLIC_KEY	secret	RS256 public key. Served at /.well-known/jwks.json.
JWT_ISSUER	config	https://api.atavishaala.in

NAME	KIND	NOTE
JWT_AUD_APP	config	atv_app
JWT_AUD_ADMIN	config	atv_admin
JWT_AUD_PARTNER	config	atv_partner
JWT_AUD_FLEET	config	atv_fleet
ACCESS_TTL_SECONDS	config	600 for app, 900 for the three portals.
REFRESH_TTL_DAYS	config	30 for app, 7 for admin, 14 for partner and fleet.
PII_ENC_KEY	secret	32-byte AES-256-GCM key, base64. Rotating it requires a re-encrypt job.
PII_INDEX_HMAC_KEY	secret	HMAC-SHA256 key for the searchable index columns. Never rotated.
TOTP_ISSUER	config	Atavishaala Admin — appears in the authenticator app.
ARGON2_MEMORY_KIB	config	19456. Tuned for the Cloud Run 512 MiB instance.
CLOUDINARY_CLOUD_NAME	config	Public. Also present in the portals.
CLOUDINARY_API_KEY	secret	Server-side only.
CLOUDINARY_API_SECRET	secret	Signing only. Must never reach any client bundle.
RAZORPAY_KEY_ID	config	Public.
RAZORPAY_KEY_SECRET	secret	Server-side only.
RAZORPAY_WEBHOOK_SECRET	secret	Signature verification on the payment webhook.
WHATSAPP_TOKEN	secret	Meta Cloud API permanent token.
WHATSAPP_PHONE_ID	config	Sender phone number id.
SMS_2FACTOR_KEY	secret	Fallback OTP provider.
BREVO_API_KEY	secret	Transactional email for invites and resets.
FCM_SERVICE_ACCOUNT	secret	Base64 service-account JSON for push.
SENTRY_DSN_API	config	Not a secret, but keep it out of the client bundles.
CRON_SHARED_SECRET	secret	Header the GitHub Actions cron presents to /internal/cron/*.
ALLOWED_ORIGINS	config	Exact list of the four portal and app origins. No wildcard.
BOOTSTRAP_ADMIN_EMAIL	config	Consumed once by M1. Remove after the first login.
OFFER_WAVE_TTL_SECONDS	config	600, 600, 900 for waves 1, 2 and 3.
RIDER_MAX_ACTIVE_JOBS	config	8.
POD_GEOFENCE_METRES	config	200.
MANIFEST_WINDOW_MINUTES	config	240. How long an unmask stays valid after OUT_FOR_DELIVERY.
COD_VARIANCE_BLOCK_BPS	config	200, i.e. 2 %.
EXPORT_ROW_CAP	config	5000.
AUDIT_DIFF_MAX_BYTES	config	8192.

11.5.1 Portal variables — the same six in all three portals

NAME	KIND	NOTE
VITE_API_BASE	public	https://api.atavishaala.in/api/v1 — the namespace is appended in code.
VITE_NAMESPACE	public	admin partner fleet. Determines the client namespace and cookie name.
VITE_SENTRY_DSN	public	Per-portal Sentry project.
VITE_POSTHOG_KEY	public	Omitted on admin: no autocapture on a PII-adjacent surface.
VITE_CLOUDINARY_CLOUD	public	For rendering signed URLs only. No key, no secret.
VITE_BUILD_SHA	public	Injected by CI. Shown in the footer for support triage.

The three portals take the same six variables and differ only in the values. That is what makes the shared kit possible and it is also what makes a mistake copy itself three times, so the portal environment is reviewed once by FE-1 in W2 and then treated as frozen. VITE_NAMESPACE is the one with teeth: it selects the API namespace

and the cookie name, and a portal deployed with the wrong value will authenticate against the wrong audience and fail every request with 401 rather than failing at build time. AA-04 section 4.2 explains why that failure mode is preferable to a portal that quietly reaches a namespace it has no business in.

VITE_ PREFIXED MEANS PUBLIC. A SECRET IN A PORTAL BUNDLE IS A SHIPPED SECRET.

Vite inlines every VITE_ variable into the JavaScript it emits. There is no such thing as a private VITE_ value, no matter what the hosting dashboard calls the field, and minification is not obfuscation — anyone can read it with view-source and thirty seconds. The only correct response to a secret found in a portal bundle is to rotate it, because it is already public. The bundle budget gate in section 11.3 will not catch this; the secret scan in CI will, and the pre-commit hook should. If a portal genuinely needs a signed value, the API signs it and the portal fetches it.

11.6 THE DEFINITION OF DONE

20 demonstrations, D-01 to D-20, all of them on the deployed hosts

ID	DEMONSTRATION
D-01	A staff user can sign in with email, password and TOTP, and cannot sign in without TOTP.
D-02	Effective permissions resolve with DENY winning, and the resolution is unit-tested against a matrix.
D-03	Every mutating admin request appears in the audit log with a before/after diff.
D-04	No screen in any portal renders more than the last four characters of a masked field.
D-05	A product company can be invited, submit seven KYC documents, be verified and go ACTIVE.
D-06	A partner-drafted product passes moderation and appears in the mobile app catalog.
D-07	A multi-supplier order splits into shipments, and the app tracker shows a correct derived status.
D-08	A supplier can accept, pack, declare a shortfall, mark ready and hand over.
D-09	A delivery company can onboard, define service areas with a rate card, and add riders.
D-10	An offer broadcasts, expires, escalates through three waves and lands on the admin board.
D-11	A rider opens the manifest, the disclosure is registered, and delivery completes with OTP inside 200 m.
D-12	Three failed attempts force RTO automatically and the customer is refunded on RTO_DELIVERED.
D-13	A weekly payout run assembles, locks, releases with a UTR, and reconciles to the paisa.
D-14	A COD period is declared by a fleet and reconciled by finance, with variance computed.
D-15	A return with photo evidence is approved, picked, received by the supplier and refunded, with a compensating ledger entry.
D-16	An agronomist claims an advisory request from the admin portal and records a callback within the SLA.
D-17	All twelve CI gates pass on all five repositories.
D-18	M1..M8 apply cleanly to a restored production snapshot and the derived-status diff is zero rows.
D-19	The monthly bill is ₹67 and the eight free-tier alert thresholds are configured and firing on test.
D-20	Eleven runbooks are written and each has been rehearsed once by someone other than its author.

These twenty lines are the programme-level acceptance criteria and they are stated as demonstrations rather than as features on purpose: each one is a thing a person performs in front of other people, in order, on the deployed hosts, using data that was not created by a test fixture. AA-10 section 10.9 explains what the programme has and has not bought when all twenty pass, and that reasoning is not repeated here. What matters for daily work is the mapping: every task in sections 11.7 to 11.16 exists because some part of one of these twenty lines cannot be demonstrated without it, and a task that cannot be traced to a D-number is a task somebody added without asking what it was for.

11.7 W1 — FOUNDATIONS AND IDENTITY

16 tasks · gate: Five repos exist, CI green on all five, staff login with TOTP works end to end, the kit publishes its first version.

Week 1 is not a setup week, whatever it looks like on the schedule. It is the week that decides whether the other nine are cheap or expensive, because everything that four people will disagree about later is fixed here: the repository boundaries, the shape of a Zod-first contract, the shape of a kit component, and the fact that a deploy is part of a task. Sixteen tasks is the highest count in the programme and most of them are small, which is the correct shape for a week where the real output is agreement rather than features.

The ordering constraint inside the week is a single dependency and it runs FE-1 first. Nothing FE-2 is scheduled to do can start until atavishaala-portal-kit publishes v0.1 with a working shell, a table, a form field and the four states, because the alternative is FE-2 building those primitives a second time in the supplier portal and then unbuilding them. The backend pair is not blocked in the same way: BE-1 can seed identity while BE-2 stands up the API skeleton and the contract package, and neither needs the other before Thursday.

The thing most likely to go wrong is the kit slipping past Wednesday. If v0.1 is not published at the Wednesday blocker sync, FE-2 is idle for two days and the week silently borrows from W2, which is the week with eighteen tasks and no slack. The mitigation is to ship the kit deliberately unfinished — four components and the states, nothing else — and let the additive-within-a-minor rule carry the rest. Friday has to show a staff user signing in with email, password and TOTP on the deployed API, TOTP refused when absent, all twelve gates green on all five repositories, and the kit version pinned exactly in one portal package.json.

WATCH FOR

The kit is the bottleneck. If FE-1 has not published v0.1 by Wednesday, FE-2 is idle.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P1-FE1-01	FE-1	repos	Create the four new repositories with a shared template	1. Create atavishaala-portal-kit, -admin-portal, -supplier-portal, -fleet-portal under the org. 2. Copy the tsconfig, eslint, prettier and .editorconfig from apps/console into a template. 3. Enable branch protection on main: one review, all checks required, no force push. 4. Add CODEOWNERS naming FE-1 for the kit and admin, FE-2 for supplier and fleet.	✓ Four repos exist with main protected. ✓ A trivial PR on each repo runs CI and reports typecheck plus lint.
P1-FE1-02	FE-1	kit	Scaffold the kit as a Vite library with a Tailwind preset	1. vite build in lib mode emitting ESM plus .d.ts; no CJS. 2. Export a tailwind preset carrying the brand tokens: green 009411, violet 8735F5, ink 14171A. 3. Set up tsup-free path aliases and a single src/index.ts barrel. 4. Publish 0.1.0 to GitHub Packages from a tag-triggered workflow.	✓ npm i @atavishaala/kit@0.1.0 succeeds from a clean machine with a PAT. ✓ The preset applied in a portal renders the brand green on a button.
P1-FE1-03	FE-1	kit	Build the primitive layer	1. Button, Input, Select, Textarea, Checkbox, Radio, Switch, Badge, Chip, Spinner, Skeleton. 2. Every interactive primitive takes a disabled and a loading state and forwards a ref. 3. Focus rings visible on keyboard focus only; contrast checked at 4.5:1 against both fills. 4. Write a Ladle or Vite-dev playground page listing every primitive and every state.	✓ Eleven primitives exported and rendered in the playground. ✓ Keyboard tab order through the playground reaches every control with a visible ring.
P1-FE1-04	FE-1	kit	Build the session and API client layer	1. createClient(namespace) reading VITE_API_BASE and appending the namespace segment. 2. In-memory access token, silent refresh on 401 with a single-flight lock, logout on refresh failure. 3. useSession() and a <RequireAuth> route wrapper. 4. Throw a typed ApiError carrying code, message and fieldErrors.	✓ A 401 triggers exactly one refresh call even with six concurrent requests in flight. ✓ grep for localStorage in the kit returns nothing.
P1-BE1-01	BE-1	backoffice	Add the backoffice module and M1	1. Create src/modules/backoffice with routes.admin.ts, service.ts, repo.ts, schema.ts. 2. Write M1 creating staff_users, staff_roles, staff_role_grants, role_permissions, staff_totp. 3. Seed the nine system roles with empty permission sets for now. 4. Create the bootstrap SUPER_ADMIN from BOOTSTRAP_ADMIN_EMAIL with totp_enrolled_at NULL.	✓ prisma migrate deploy applies M1 to a fresh branch with no manual step. ✓ SELECT count(*) FROM staff_roles returns 9.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P1-BE1-02	BE-1	identity	Split the token audiences	1. Add JWT_AUD_ADMIN, _PARTNER, _FLEET and mint with the correct aud per login path. 2. Write requireActor(kind) asserting the aud claim matches the mounted namespace. 3. Mount four routers: /api/v1/app, /admin, /partner, /fleet, each with its own actor guard. 4. Add a namespace-mismatch integration test for all twelve wrong combinations.	✓ An app token on /admin/me returns 401 WRONG_AUDIENCE, not 403. ✓ The twelve-combination test passes.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P1-BE1-03	BE-1	backoffice	Staff password and TOTP login	1. Argon2id hashing at m=19456 t=2 p=1; timing-safe compare; generic error text on failure. 2. Login returns a short-lived challenge token, never a session, when TOTP is pending. 3. POST /admin/auth/totp verifies a 30-second window with a one-step skew and stores last_used_step. 4. Lock the account for 15 minutes after ten failed attempts, counting both factors.	✓ Login without a TOTP factor cannot produce a session; it forces enrolment. ✓ Replaying the same six digits inside one window returns 401 TOTP_REPLAY.
P1-BE1-04	BE-1	identity	Portal refresh cookies and rotation	1. Issue atv_admin, atv_partner, atv_fleet as Secure, HttpOnly, SameSite=Strict, host-only cookies. 2. Reuse of a rotated refresh token revokes the entire family and writes a security event. 3. Set ALLOWED_ORIGINS to the exact four origins; reject any other Origin with 403. 4. TTLs: 7 days admin, 14 days partner and fleet.	✓ A replayed refresh token invalidates the family and the next request returns 401. ✓ A request from an unlisted origin is refused before any handler runs.
P1-BE2-01	BE-2	orders	Design and review the shipment split on paper	1. Write the shipment FSM table and the reduceOrderStatus reduction into the repo as a doc test. 2. Enumerate all 28 shipment-state combinations with the expected order status. 3. Walk BE-1 and FE-1 through it in the Wednesday sync before writing any code. 4. Record the decision that orders.status becomes derived and never written.	✓ A table-driven test file exists with 28 cases, all currently skipped. ✓ The reduction is agreed in writing by three people before W4 starts.
P1-BE2-02	BE-2	shared	Extract assertTransition into a reusable FSM helper	1. Move the existing order transition table into packages/shared/fsm.ts as a generic. 2. assertTransition(machine, from, to, actorKind) throws a typed InvalidTransition error. 3. Register machines for order, shipment, delivery_job, advisory, kyc, payout_run, cod. 4. Add the aa/no-status-write ESLint rule and fix every existing violation.	✓ Seven machines registered; each has a from-set, to-set and permitted actor kinds. ✓ eslint reports zero aa/no-status-write violations in apps/api.
P1-BE2-03	BE-2	platform	Idempotency middleware	1. Create idempotency_keys with UNIQUE (actor_kind, actor_id, endpoint, key) and a 24 h TTL. 2. Middleware applied by route metadata, not per handler; missing header returns 400. 3. Store the response status and body; a replay returns them byte for byte. 4. A different request body under the same key returns 409 IDEMPOTENCY_MISMATCH.	✓ Replaying a money fixture twice produces one ledger row and two identical responses. ✓ The idempotency-gc cron deletes expired keys.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P1-FE2-01	FE-2	portals	Scaffold the supplier and fleet portals	1. Vite React TS in both repos, Router 6, TanStack Query, the kit at an exact version. 2. Wire VITE_NAMESPACE to partner and fleet respectively; one client per portal. 3. Add the login and a placeholder dashboard route behind RequireAuth. 4. Connect both repos to Cloudflare Pages with a preview deployment per PR.	✓ seller and fleet preview URLs deploy from a PR and render the login screen. ✓ Both package.json files pin the kit without a caret.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P1-FE2-02	FE-2	portals	Portal security headers and CSP	1. Add _headers on each Pages project: CSP, HSTS, X-Frame-Options DENY, Referrer-Policy. 2. CSP connect-src lists exactly api.atavishaala.in and res.cloudinary.com. No wildcard. 3. No unsafe-inline; move the one inline style into the stylesheet. 4. Write a Playwright check asserting the header on the deployed host.	✓ curl -I on each host shows the CSP with no unsafe-inline. ✓ The Playwright check fails the build if a header is missing.
P1-FE1-05	FE-1	admin	Scaffold the admin portal and its login flow	1. Same scaffold as the two portals, with VITE_NAMESPACE=admin and no PostHog key. 2. Build AD-01, AD-02, AD-03: password, TOTP challenge, enrolment with recovery codes. 3. Recovery codes are shown once, with a copy button and a confirmation checkbox. 4. Deploy to admin.atavishaala.in with Cloudflare Access disabled but the WAF rule on.	✓ A real staff user completes enrolment and reaches an empty dashboard. ✓ Reloading the enrolment screen does not show the recovery codes again.
P1-BE1-05	BE-1	platform	Structured logging and PII log discipline	1. Add a Pino redact list covering every masked field name plus authorization and cookie. 2. Log the actor kind, actor id, permission and target id; never a value. 3. Add a Sentry beforeSend that drops any breadcrumb matching the ten-digit pattern. 4. Add the CI grep for ten consecutive digits over the api and portal source trees.	✓ A deliberate log of a phone number appears as [Redacted] in the emitted line. ✓ The CI grep fails on a test commit containing a ten-digit literal.
P1-ALL-01	all	process	Agree the working agreement and open the gate log	1. Adopt the twelve conventions in AA-11 Appendix D as written. 2. Create a gate log document with one row per week, to be marked passed or failed. 3. Book the three recurring syncs in calendars for all ten weeks. 4. Agree that a failed gate triggers a cut-list decision at the same meeting.	✓ The gate log exists with ten empty rows. ✓ Three recurring invitations are accepted by everyone.

11.8 W2 — PERMISSIONS, AUDIT AND THE ADMIN SHELL

18 tasks · gate: The 88 permissions are seeded and enforced; every admin mutation is audited; the admin shell renders 12 read-only screens against real data.

Week 2 builds the authorisation substrate that every later week assumes exists, and it is the largest week by task count for a reason: the eighty-eight permission rows, the DENY-wins resolver, the audit middleware and the masking extension are four separate pieces of work that only become useful simultaneously. A permission matrix without a resolver is a table nobody reads. A resolver without audit logging is unprovable. Masking that arrives after the first list screen means the first list screen leaked, and unmasking a screen retroactively is a code change plus an incident note.

The ordering inside the week is strict and it is not the order the tasks are numbered in. The resolver and the masking extension both land before any admin list screen renders a single row of real data. That means BE-1 owes FE-1 a working GET /admin/me with effective permissions by mid-week, and FE-1 builds the twelve read-only screens against that shape rather than against a mock, because a mock that returns a flat array of permission strings will not survive contact with DENY-wins.

The failure to watch for is subtler than a missing feature: it is a resolver that is correct for the nine seeded roles and wrong for a user who holds two of them. Resolution order, the union of grants and the precedence of a single DENY over any number of ALLOWs are exactly the cases a hand-written test suite skips. AA-08 section 8.3 gives the matrix; the unit test drives all of it, not a sample. Friday shows twelve admin screens against real Neon data, an audit log entry with a before and after diff for every mutation performed during the demo, and a masked column that shows four characters with no affordance to reveal more.

WATCH FOR

DENY-wins resolution and the masking extension must both land before any list screen.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P2-BE1-01	BE-1	backoffice	Define the 88-permission union and seed the role sets	1. Write the Permission type union in <code>packages/shared/permissions.ts</code>. There is no <code>pii.read.raw</code>. 2. Seed <code>role_permissions</code> for the nine system roles from a checked-in TypeScript fixture. 3. Write <code>resolveEffective(staffUserId)</code> returning a Set, with DENY winning across all held roles.	✓ A staff user holding <code>OPS_MANAGER</code> and a DENY on <code>order.cancel</code> cannot cancel. ✓ A unit matrix covers 88 permissions across the nine roles plus two DENY overlays.
P2-BE1-02	BE-1	backoffice	<code>requirePermission</code> and the route permission registry	1. <code>requirePermission(perm)</code> reads the resolved set from the request context, cached per request. 2. Every admin route declares its permission in route metadata, not in the handler body. 3. A CI sweep asserts every admin route declares a permission that exists in the union.	✓ A route without a declared permission fails CI, not runtime. ✓ <code>GET /admin/me</code> returns the flattened effective array the portal uses to gate UI.
P2-BE1-03	BE-1	backoffice	Step-up factor freshness	1. <code>POST /admin/auth/step-up</code> records a fresh factor timestamp on the session. 2. <code>requireFreshFactor(300)</code> guards the twenty-four mutating step-up routes listed in AA-04. 3. A stale factor returns 403 <code>STEP_UP_REQUIRED</code> with the seconds since the last factor.	✓ Approving a refund 6 minutes after step-up returns 403 with a machine-readable code. ✓ The kit intercepts that code and opens the TOTP modal without losing the form state.
P2-BE1-04	BE-1	platform	The PII masking Prisma extension	1. Add the three-column pattern helpers: <code>encrypt</code>, <code>last4</code>, <code>hmacIndex</code> in <code>packages/shared/pii.ts</code>. 2. A Prisma client extension strips every <code>*_cipher</code> field from every result, unconditionally. 3. <code>unmaskAddress()</code> is the single exported escape hatch and is not exported from the barrel.	✓ A raw <code>prisma.users.findMany()</code> in a REPL returns no cipher column at all. ✓ <code>grep</code> for <code>_cipher</code> outside <code>packages/shared/pii.ts</code> and the manifest handler returns nothing.
P2-BE1-05	BE-1	platform	Audit middleware with before/after diff	1. Mount audit as router-level middleware on <code>/admin</code> for every method other than GET. 2. Capture the target row before the handler and after, then diff, cap at <code>AUDIT_DIFF_MAX_BYTES</code>. 3. Diff values pass through the masking extension before they are stored.	✓ Every mutating fixture increases <code>audit_log</code> by exactly one row. ✓ A diff containing a phone number stores only the last four.
P2-BE1-06	BE-1	platform	PII disclosure register and <code>discloseOrThrow</code>	1. Create <code>pii_disclosures</code> in M8 and write <code>discloseOrThrow(actor, subject, field, reason, ref)</code>. 2. The register row and the unmasked read happen in the same transaction or neither happens. 3. Add <code>GET /admin/pii-disclosures</code> behind <code>audit.read</code> with <code>actor</code>, <code>subject</code> and <code>date</code> filters.	✓ A failed transaction leaves no disclosure row and no unmasked value returned. ✓ The register is queryable by subject user id in under 200 ms with the index in place.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P2-FE1-01	FE-1	kit	The permission gate and masked-field components	1. <Can perm="x"> and useCan() reading the effective array from useSession. 2. <Masked value last4 /> renders a dot sequence plus the last four and has no reveal prop. 3. Add the aa/no-row-pii-render ESLint rule to the kit's shared config.	✓ There is no code path in the kit that can render a full masked value. ✓ A portal component that interpolates a phone field directly fails lint.
P2-FE1-02	FE-1	kit	The data layer: table, filters, saved views	1. <DataTable> with server-side pagination, column visibility and a sticky header. 2. <FilterBar> producing a typed query object; <SavedViews> reading and writing /admin/saved-views. 3. An empty state, an error state and a skeleton loading state for every table.	✓ A 5 000-row list scrolls at 60 fps with 25 rows per page and no layout shift. ✓ Switching a saved view changes the URL so the state is shareable.
P2-FE1-03	FE-1	kit	The shell: navigation, command palette, toasts, drawers	1. <AppShell> with a collapsible sidebar filtered by permission, breadcrumbs and a user menu. 2. A command palette on Ctrl-K jumping to any screen the user is permitted to see. 3. <Drawer>, <Modal>, <ConfirmDialog> with a typed reason field, and a toast queue.	✓ A user without fleet.read never sees the Fleets nav entry, not even disabled. ✓ The palette lists only permitted destinations, verified by a test with two role sets.
P2-FE1-04	FE-1	admin	Admin read-only screens, first tranche	1. Build AD-04 dashboard shell, AD-05 order list, AD-06 order detail, AD-10 job list. 2. Wire the customer column through <Masked> only. No detail view shows a full phone. 3. Add a build-time check that fails if a screen imports anything but the kit client.	✓ Four screens render against the real API with pagination and filters working. ✓ A reviewer cannot find a full phone number anywhere in the four screens.
P2-FE1-05	FE-1	admin	Admin read-only screens, second tranche	1. Build AD-15 product list, AD-16 product detail, AD-21 partners, AD-25 fleets, AD-39 customers. 2. Each list gets a saved view and an empty state with a next action. 3. Add AD-41 audit and disclosures as two tabs over the two read endpoints.	✓ Nine admin screens total are usable read-only against production-shaped data. ✓ The audit tab renders a diff in a readable two-column form.
P2-BE2-01	BE-2	orders	Write M4 and rehearse it on a restored branch	1. Write M4 creating shipments, shipment_items and order_events with the backfill in one transaction. 2. Restore a Neon branch from a snapshot and apply M1..M4; record the duration. 3. Write a verification query diffing stored orders.status against reduceOrderStatus for every row.	✓ The verification query returns zero rows on the restored branch. ✓ The measured duration is recorded and is under the 15-minute read-only window.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P2-BE2-02	BE-2	orders	Implement reduceOrderStatus and enable the 28 cases	1. Implement the pure reduction in packages/shared/orderStatus.ts with no database access. 2. Un-skip the 28 table-driven cases from P1-BE2-01 and make them pass. 3. Add recomputeOrder(orderId) as the only writer of orders.status.	✓ 28 of 28 cases pass. ✓ grep for orders.*status inside an update outside recomputeOrder returns nothing.
P2-BE2-03	BE-2	fleet	Write M5 and M6 and seed the internal fleet	1. M5 creates the fleet tenancy tables; M6 creates delivery_jobs, job_offers and pod_records. 2. Seed ATV-SELF as a tier-1 fleet covering the current serviceable pincode set. 3. M6 backfills a legacy delivery_job per existing shipment, flagged legacy_backfill.	✓ A fresh branch applies M1..M6 with no manual intervention. ✓ ATV-SELF exists with at least one service area and can be offered a job.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P2-BE2-04	BE-2	money	Refund approval workflow with two-person control	1. POST /admin/refunds creates PENDING_APPROVAL and never touches the gateway. 2. POST /admin/refunds/:id/approve requires refund.approve, a fresh factor and a different staff id. 3. Approval routes to Razorpay or to wallet_ledger; both paths are idempotent.	✓ The initiator cannot approve their own refund; the API returns 403 SAME_ACTOR. ✓ A replayed approval produces one gateway call and one ledger row.
P2-FE2-01	FE-2	supplier	Supplier auth, invite acceptance and onboarding checklist	1. Build SP-01, SP-02, SP-03 against the partner auth endpoints. 2. The checklist blocks every selling action until profile, KYC and bank are submitted. 3. Show org status prominently: DRAFT, KYC_SUBMITTED, KYC_VERIFIED, ACTIVE, SUSPENDED.	✓ A fresh invite token walks to a usable session in under two minutes. ✓ A DRAFT org sees the checklist, not the dashboard, on every route.
P2-FE2-02	FE-2	fleet	Fleet auth with the dual login mode	1. Build FL-01 with a password mode and a mobile-OTP rider mode on one screen. 2. Build FL-02 invite acceptance and FL-03 the six-step onboarding checklist. 3. A FLEET_RIDER session lands on FL-11 and never sees the desktop navigation.	✓ A rider logs in with OTP and reaches the rider stack with no sidebar. ✓ An owner logs in with password plus TOTP and reaches the control room.
P2-ALL-02	all	process	Turn on the eight ESLint rules and the twelve CI gates	1. Implement the eight custom rules in a shared eslint-plugin inside the kit repo. 2. Wire all twelve gates from AA-01 into the five workflows. 3. Fix every existing violation rather than adding a suppression.	✓ CI is green on all five repos with zero eslint-disable comments for the eight rules. ✓ A deliberately bad PR is blocked by the correct gate, verified once per gate.

11.9 W3 — PARTNER TENANCY AND THE MODERATION PIPELINE

17 tasks · gate: A product company can be invited, submit KYC, be verified, draft a product and submit it; a catalog manager can approve it and it appears in the app.

Week 3 is the first week that produces something a person outside the team can recognise as a product. It also introduces the concept that the rest of the programme is organised around: a tenant. Until now every row in the database belonged to the house. From M2 onward every product has an owning organisation, tenant scope arrives only from a verified token claim, and a request that carries an organisation identifier in its body or query string is rejected with 400 SCOPE_IN_REQUEST rather than helpfully honoured. That rejection is not defensive programming; it is the only way to prove that no code path takes scope from user input.

The dependency chain here is longer than in either previous week and it crosses two repositories. M2 backfills every existing product to ATV-HOUSE, M3 then makes the column NOT NULL and adds moderation_status, and only after M3 can the moderation queue be built against a real index. On the front end FE-2 needs partner authentication working before any supplier screen is testable, so BE-1 ships partner login and invitation acceptance ahead of the KYC upload screens even though KYC is the more visible piece.

What usually goes wrong in this week is the moderation state machine, specifically the transition out of REJECTED. A product that is rejected, edited and resubmitted must re-enter IN_REVIEW rather than jumping to APPROVED on the strength of its earlier approval, and the default flip in M3 — APPROVED for existing rows, DRAFT for new ones — makes it easy to write a query that treats the two populations as one. Friday is the first live cross-portal demonstration in the programme and it has to run in one sitting: invite a company from the admin portal, accept the invitation, upload the seven KYC documents, verify, draft a product, submit it, approve it as a catalog manager, and see it in the mobile app catalog.

WATCH FOR

First end-to-end cross-portal flow. Demo it live at the Friday gate.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P3-BE1-01	BE-1	partners	Apply M2 and M3 and make partner_org_id mandatory	1. M2 adds the tenancy tables and a nullable products.partner_org_id, backfilled to ATV-HOUSE. 2. M3 sets the column NOT NULL, adds moderation_status and flips the default to DRAFT. 3. Add the partial index on products (partner_org_id, moderation_status).	✓ Every existing product belongs to ATV-HOUSE and is APPROVED. ✓ A new product created without a partner_org_id is rejected by the database, not the code.
P3-BE1-02	BE-1	partners	The scoped() repo wrapper and the tenancy guard	1. scoped(prisma, principal) returns a client extension injecting the org where clause. 2. Every repo function in a partner or fleet namespace takes the scoped client, never the raw one. 3. Add aa/no-tenant-in-input and reject a body carrying partner_org_id with 400 SCOPE_IN_REQUEST.	✓ A crafted request with another org's id in the body is refused before the handler. ✓ A partner reading /partner/products cannot see a foreign product even by guessing an id.
P3-BE1-03	BE-1	partners	Partner onboarding, invitation and KYC submission	1. POST /admin/partners creates a DRAFT org and a single-use 72-hour owner invite via Brevo. 2. POST /partner/kyc accepts seven document types via a signed Cloudinary upload to a private folder. 3. Replacing a document resets it to SUBMITTED and reopens the org KYC state.	✓ An invited owner sets a password and uploads all seven documents. ✓ The Cloudinary secret does not appear in any portal bundle, verified by grep on dist.
P3-BE1-04	BE-1	partners	KYC review, activation and suspension	1. POST /admin/partner-kyc/:id/review with VERIFY or REJECT and a mandatory note on reject. 2. When all mandatory documents are VERIFIED the org moves to KYC_VERIFIED automatically. 3. POST /admin/partners/:id/status with step-up; SUSPENDED delists every SKU in one statement.	✓ Suspending an org removes its products from the app catalog within one request. ✓ The org status machine rejects ACTIVE from DRAFT without passing through KYC_VERIFIED.
P3-BE1-05	BE-1	catalog	The moderation pipeline	1. A partner edit to an APPROVED product forks a draft revision; the live row is untouched. 2. POST /partner/products/:id/submit moves DRAFT to IN_REVIEW and enters the admin queue. 3. POST /admin/products/:id/moderate applies APPROVE, REJECT or DELIST with a reason.	✓ Approving a revision replaces the live row atomically and records moderation history. ✓ Rejecting returns the draft to the partner with the verbatim reviewer note visible.
P3-BE1-06	BE-1	partners	Rate cards, bank accounts and commission	1. partner_rate_cards is effective-dated; publishing a new rate supersedes, never edits. 2. PUT /partner/bank stores cipher plus last four and enters PENDING_VERIFICATION, pausing payouts. 3. POST /admin/partner-bank/:id/verify with step-up sets verified_at and resumes payouts.	✓ Querying the commission for a past order returns the rate that was effective then. ✓ A partner with an unverified bank account accrues payouts but cannot be included in a run.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P3-FE1-01	FE-1	admin	Partner management screens	1. Build AD-21, AD-22 with six tabs, AD-23 invite and AD-24 KYC review. 2. The KYC viewer streams a 5-minute signed URL, disables right-click download and logs the view. 3. Bank and GSTIN fields render through <Masked> with no reveal control.	✓ A reviewer verifies a real document set end to end from the admin portal. ✓ Reloading the KYC viewer after five minutes forces a new signed URL and a new disclosure row.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P3-FE1-02	FE-1	admin	Moderation and catalog screens	1. Build AD-18 with a side-by-side diff of proposed against live, AD-17 the six-step wizard. 2. Build AD-19 category tree with drag-to-reparent and AD-20 stock adjustments. 3. Reject requires a reason of at least twenty characters, enforced in the form and the API.	✓ A catalog manager approves a partner product and it appears in the app within a minute. ✓ The diff highlights changed attributes, media additions and price deltas distinctly.
P3-FE1-03	FE-1	kit	Media components and the signed upload flow	1. <Uploader> requesting a signature from the API, compressing client-side, then posting direct. 2. Image compression to a 1 600 px long edge and 80 % quality before upload. 3. <MediaGallery> with a lightbox, and <DocumentViewer> for KYC with download disabled.	✓ A 6 MB phone photo uploads as roughly 400 KB. ✓ The uploader never sees an API secret; only a signature, a timestamp and a folder.
P3-FE2-01	FE-2	supplier	Supplier catalog screens	1. Build SP-10 my products, SP-11 the new draft wizard, SP-12 detail and SP-13 edit. 2. The wizard collects the compliance licence number for seed, pesticide and fertiliser categories. 3. Show the moderation chip and the reviewer note prominently on a rejected draft.	✓ A supplier creates a compliant draft with three variants and four images and submits it. ✓ An approved product edited by the supplier does not change what customers see until approved.
P3-FE2-02	FE-2	supplier	Supplier stock, pricing and moderation history	1. Build SP-14 the editable stock grid with ETag conflict detection and a CSV paste box. 2. Build SP-15 price requests and SP-16 review history. 3. A conflicting stock write shows the current server value and asks for a decision.	✓ Two browser tabs editing the same variant produce a visible conflict, not a silent overwrite. ✓ A CSV paste of 200 rows updates stock in one request with a per-row result list.
P3-FE2-03	FE-2	supplier	Supplier settings and KYC screens	1. Build SP-23 bank, SP-24 KYC with seven slots, SP-25 team and SP-26 organisation profile. 2. Bank and GSTIN render masked after save; the input is write-only. 3. The team screen refuses to remove the last PARTNER_OWNER in the UI and in the API.	✓ A supplier owner invites a catalog seat and a finance seat with correct permissions. ✓ The KYC screen shows per-document status and the reviewer note for each rejection.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P3-BE2-01	BE-2	orders	The shipment splitter	1. splitOrder(orderId) groups order_items by partner_org_id and creates one shipment per group. 2. Called at payment capture for prepaid and at COD confirmation for cash orders. 3. Sum of shipment_items.qty per order_item must equal the ordered quantity, asserted in the transaction.	✓ A cart with items from three suppliers produces exactly three shipments. ✓ The assertion fails loudly, not silently, if a quantity is lost.
P3-BE2-02	BE-2	orders	The order event timeline	1. Write order_events on every shipment transition, payment event and refund event. 2. The app tracker reads only order_events, never shipments, so its contract does not change. 3. Add a customer-visible flag so internal events never surface in the app.	✓ The existing app tracker renders correctly against a three-shipment order with no app change. ✓ An internal force-transition appears in the admin timeline and not in the app.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P3-BE2-03	BE-2	fleet	Fleet onboarding, service areas and riders	1. POST /admin/fleets and the owner invite mirror the partner flow, adding transport licence and insurance. 2. POST /fleet/areas/request accepts a bulk pincode set with per-pincode fee, COD flag and weight cap. 3. POST /fleet/riders creates a PENDING_DOCS rider; documents move it to ACTIVE after review.	✓ A fleet defines 40 pincodes in one request and sees them pending approval. ✓ A rider without documents cannot be attached to a job.
P3-BE2-04	BE-2	fleet	Serviceability resolution	1. resolveFleets(pincode, weight, cod) returns eligible fleets ordered by tier then priority. 2. GET /admin/serviceability/:pincode exposes it for the ops gap view. 3. The app checkout calls it to refuse an unserviceable pincode before payment.	✓ An unserviceable pincode is refused at checkout with a clear message, not at fulfilment. ✓ A 30 kg equipment order resolves only to fleets with a PICKUP or TRUCK vehicle.
P3-ALL-03	all	demo	Cross-portal end-to-end rehearsal	1. Invite a test product company, verify KYC, draft a product, approve it, buy it in the app. 2. Walk the whole path live at the Friday gate on the deployed hosts. 3. Log every friction point as a task for W4 or the backlog, with an owner.	✓ The full path completes without a developer touching the database. ✓ The friction list exists and is triaged before Monday.

11.10 W4 — THE SHIPMENT SPLIT

15 tasks · gate: Orders split into shipments per supplier; orders.status is derived; the app tracker still reads correctly; M4 is proven on a restored snapshot.

Week 4 is the week the programme is most likely to lose, and the schedule reflects that by putting nothing else on BE-2. Splitting an order into per-supplier shipments changes the meaning of a row that already exists in production and that the mobile app already reads. There is no version of this work that is safe because it is small; it is safe only because it is rehearsed. Fifteen tasks sounds like a normal week, but four of them are the same migration approached from four different directions: write it, rehearse it, prove the derived status, and prove the app tracker.

The order of operations is not negotiable. The shipment FSM and assertTransition land before M4, because a backfill that writes states the FSM would reject produces rows that no later code path can move. reduceOrderStatus() and its twenty-eight-combination table test land before the app tracker change, because MOB-1 needs a status contract that will not shift under them. M4 itself runs against a restored Neon branch and produces a derived-status diff of zero rows before it is applied anywhere that a human will look at.

The specific failure to fear is a silent one. orders.status is derived and never written, so a shipment left in a state that maps to the wrong order status does not raise an error anywhere — it simply shows a customer the wrong thing in the app, and nobody notices until a support call arrives weeks later. That is why the diff pass condition is zero rows rather than a small explicable number. Friday shows a multi-supplier order splitting correctly, the app tracker reading the derived status for both the old backfilled orders and new ones, and the rehearsal output pasted into the demo notes with its zero-row diff.

WATCH FOR

The single riskiest week. Nothing else is scheduled in parallel for BE-2.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P4-BE2-01	BE-2	orders	Ship M4 to production behind a read-only window	1. Take a Neon branch immediately before the window as the rollback point. 2. Put the API in read-only mode for 15 minutes via a feature flag that 503s writes with Retry-After. 3. Apply M4, run the verification diff, then lift the flag only if the diff is zero rows.	✓ The diff is zero rows in production and the window closed inside 15 minutes. ✓ RB-11 rollback was rehearsed on the branch before the real window opened.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P4-BE2-02	BE-2	orders	Make orders.status derived in production	1. Keep orders.legacy_status populated for two releases as a safety net. 2. recomputeOrder runs inside every shipment transition transaction, never asynchronously. 3. Add a nightly consistency job comparing stored against derived and alerting on any drift.	✓ The nightly job reports zero drift for seven consecutive nights. ✓ No endpoint in any namespace writes orders.status; verified by the ESLint rule and a grep.
P4-BE2-03	BE-2	orders	Shipment FSM endpoints for the partner namespace	1. Implement accept, pack, ready, short-ship and handover, each idempotent and scoped. 2. accept validates ready_by against the supplier cut-off and refuses a time past the promise. 3. short-ship reduces shipment_items, releases stock and queues a partial refund.	✓ Sixteen shipment states with their permitted actors are enforced by assertTransition. ✓ A short-ship of two units out of five refunds exactly the two units plus proportional shipping.
P4-BE2-04	BE-2	orders	Cancellation and stock release across shipments	1. Cancelling one shipment releases only that shipment's reserved stock, with the inverse statement. 2. Cancelling the last live shipment cancels the order and reverses the payment or COD. 3. A cancellation after HANDED_OVER is refused; it must go through RTO.	✓ Stock returns to the exact pre-order value after a full cancellation, verified by a test. ✓ The API refuses a post-handover cancellation with 409 CUSTODY_MOVED.
P4-BE1-01	BE-1	catalog	Stock reservation across suppliers	1. Keep the single-statement conditional decrement; it now runs once per shipment group. 2. A partial failure rolls back the whole order; no order exists with a half-reserved cart. 3. Add a stock_reservations audit view so ops can see what a live order holds.	✓ Two concurrent orders for the last unit produce exactly one success and one 409 OUT_OF_STOCK. ✓ No negative stock_qty is reachable, proven by a 200-thread concurrency test.
P4-BE1-02	BE-1	backoffice	Saved views and async masked export	1. saved_views stores a Zod-allowlisted query object per staff user, optionally shared. 2. POST /admin/exports queues a job_queue row; the worker streams masked CSV to Cloudinary private. 3. The download is a 15-minute signed URL; every export writes an audit row and a disclosure row.	✓ An export of 5 000 orders completes in under 90 seconds and contains no full phone number. ✓ Requesting more than EXPORT_ROW_CAP rows returns 400 with the cap in the message.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P4-BE1-03	BE-1	platform	Settings, feature flags and the eleven dashboard tiles	1. Typed settings with a Zod validator per key; PUT requires step-up and is audited. 2. Feature flags readable at request start with a 30-second in-process cache. 3. One SQL query produces the eleven dashboard tiles; cache 30 seconds at the edge.	✓ The dashboard renders in under 400 ms at p95 with a cold cache. ✓ Flipping a flag takes effect within 30 seconds without a deploy.
P4-FE1-01	FE-1	admin	Shipment board and order detail rework	1. Rebuild AD-06 to show shipment cards with per-shipment state and supplier. 2. Build AD-07 as a kanban by shipment state with a ready_by breach lane. 3. Build AD-08 with the force-transition drawer requiring a twenty-character reason.	✓ A three-shipment order is legible on one screen without scrolling horizontally. ✓ The breach lane is populated by the same query the sla-watch cron uses.
P4-FE1-02	FE-1	admin	Dashboard, exports and settings screens	1. Build AD-04 with the eleven tiles, each linking to a pre-filtered list. 2. Add the export flow: queue, poll, download, with a visible row cap. 3. Build the settings and feature-flag screen behind settings.write with step-up.	✓ Every tile is clickable and lands on a list already filtered to that tile's meaning. ✓ An export in progress survives a page reload because the job id is in the URL.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P4-FE1-03	FE-1	kit	Kit v0.4: forms, reason fields and step-up interception	1. A form layer over react-hook-form plus Zod resolvers sharing the API schemas. 2. <ReasonField> enforcing a minimum length with a live counter. 3. The client intercepts 403 STEP_UP_REQUIRED, opens a TOTP modal and replays the request.	✓ A step-up during a long form does not lose a single entered field. ✓ Field-level API errors map onto form fields automatically from ApiError.fieldErrors.
P4-FE2-01	FE-2	supplier	Supplier fulfilment screens	1. Build SP-05 shipments to fulfil with an accept-all bulk action and a ready-by picker. 2. Build SP-06 detail and SP-07 pack-and-confirm with per-line ticking and shortfall entry. 3. The drop panel shows locality and pincode only; there is no place a phone could render.	✓ A warehouse user accepts six shipments and packs one with a shortfall in under three minutes. ✓ A code review confirms no customer contact field is fetched by these screens at all.
P4-FE2-02	FE-2	supplier	Labels, pick lists and the gate handover screen	1. Build SP-08 as a print-optimised A4 view with a Code-128 shipment barcode. 2. Build SP-09 as a mobile-first scan or type screen for the gate. 3. Print styles hide navigation and force page breaks between shipments.	✓ Six labels print on six pages from one click with no clipped barcode. ✓ The gate screen works one-handed on a 360 px viewport.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P4-FE2-03	FE-2	supplier	Supplier dashboard	1. Build SP-04 with seven tiles: to accept, breaching, low stock, in moderation, next payout, returns, rating. 2. Each tile links to a pre-filtered list. No tile shows a number without a link. 3. Show a suspension or KYC-incomplete banner above everything when applicable.	✓ A supplier can answer “what do I do next” from the dashboard alone. ✓ A suspended org sees the banner and cannot reach a mutating screen.
P4-MOB1-01	MOB-1	mobile	Order tracker adaptation	1. Read the tracker from order_events instead of inferring from status transitions. 2. Render a per-shipment sub-tracker when an order has more than one shipment. 3. Handle PARTIALLY_DELIVERED as a first-class status with clear copy in all four languages.	✓ A three-shipment order shows three sub-trackers and one overall status. ✓ The four language files contain the new strings and no key is missing.
P4-ALL-04	all	demo	Prove the split in production	1. Place a real multi-supplier order on production and walk it to DELIVERED manually. 2. Verify the app, the admin portal and the supplier portal all agree at every step. 3. Confirm the nightly consistency job reports zero drift.	✓ Three surfaces agree on the same order at six checkpoints. ✓ The gate log records W4 as passed with the evidence attached.

11.11 W5 — OFFER ALLOCATION AND THE FLEET MARKETPLACE

14 tasks · gate: A ready shipment broadcasts an offer; a delivery company accepts it, attaches a rider and a vehicle; unaccepted jobs escalate through three waves onto the admin board.

Week 5 introduces the only genuinely distributed decision in the system: which delivery company gets offered a shipment, and what happens when nobody takes it. Everything before this week has been a request arriving and a row changing. Allocation is a timed, multi-party negotiation that has to be correct when two fleets accept the same offer four milliseconds apart, and it has to terminate when none of them accepts at all. The three-wave escalation exists so that termination is a defined outcome rather than a shipment sitting unnoticed in a queue.

Inside the week, the allocator and the expiry sweep are one piece of work and should not be split across two engineers. The fleet tenancy migration M5 comes first because service areas and rate cards decide who is eligible for an offer, and the offer table cannot be indexed sensibly until fleet_service_areas exists. FE-2 can start the fleet onboarding screens against M5 while the allocator is still being written, which is why the front-end and back-end halves of this week are unusually independent for a fortnight in the middle of the programme.

Two things break here reliably. The first is fairness: an allocator that sorts by tier and then by creation time will hand every offer in a service area to the same fleet forever, which looks correct in a test with one fleet and is unacceptable with three. The second is the expiry sweep running on a cron that is not idempotent, so a slow run overlaps its successor and a wave escalates twice. Acceptance is guarded by an idempotency key for exactly this reason. Friday has to show an offer broadcast, a fleet accepting it and attaching a rider and a vehicle, and a second shipment left deliberately unaccepted so that all three waves expire in front of the room and the job lands on the admin exception board.

WATCH FOR

Allocation fairness and the expiry sweep are the two things to hammer in testing.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P5-BE2-01	BE-2	fleet	The offer allocation engine	1. On <code>READY_FOR_PICKUP</code>, create the <code>delivery_job</code> and broadcast wave 1 to tier-1 fleets in the pincode. 2. Within a wave, order by least-recently-offered, not by score, until there is real volume. 3. Accept is a single conditional <code>UPDATE</code>; losers receive <code>409 OFFER_ALREADY_TAKEN</code> and their offers go <code>WITHDRAWN</code>.	✓ Two fleets accepting within the same millisecond produce one winner and one <code>409</code>. ✓ A fleet blocked by a negative ledger balance is excluded from every wave.
P5-BE2-02	BE-2	fleet	The offer expiry sweep cron	1. Every two minutes, expire offers past <code>expires_at</code> and advance the job to the next wave. 2. Wave TTLs come from <code>OFFER_WAVE_TTL_SECONDS: 600, 600, 900</code>. 3. After wave 3 the job goes <code>UNASSIGNED</code> and raises an ops alert.	✓ A job with no acceptance reaches <code>UNASSIGNED</code> in roughly 35 minutes, not sooner or later. ✓ The sweep is idempotent; running it twice in the same minute changes nothing.
P5-BE2-03	BE-2	fleet	Manual assignment and rebroadcast	1. <code>POST /admin/jobs/:id/assign</code> moves <code>UNASSIGNED</code> to <code>ASSIGNED_MANUAL</code> then <code>ACCEPTED</code> for that fleet. 2. <code>POST /admin/jobs/:id/rebroadcast</code> takes a wave selector including <code>ALL</code>. 3. Both are idempotent and audited, and cancel is step-up guarded.	✓ An ops user clears a stuck job in two clicks from AD-11. ✓ Rebroadcasting a job already accepted returns <code>409 ALREADY_ACCEPTED</code>.
P5-BE1-01	BE-1	catalog	Category compliance and licence gating	1. Categories carry a <code>required_licence</code> enum; a draft in that category needs a matching partner licence. 2. Submitting without the licence returns <code>400 LICENCE_REQUIRED</code> naming the document type. 3. An expiring licence within 30 days raises a partner-facing warning and an ops alert.	✓ A pesticide product cannot be submitted by a partner without a verified pesticide licence. ✓ The <code>doc-expiry-watch</code> cron produces the warning at exactly 30 days.
P5-BE1-02	BE-1	partners	Partner metrics: on-time rate and acceptance latency	1. Compute a rolling 30-day on-time rate and median accept latency per partner org. 2. Materialise nightly into a small metrics table rather than computing on read. 3. Expose on AD-21 and on the supplier dashboard so both sides see the same number.	✓ The same on-time figure appears in the admin list and the supplier dashboard. ✓ The nightly job runs in under 20 seconds on the current data volume.
P5-BE1-03	BE-1	platform	Notification templates for the new audiences	1. Register WhatsApp and email templates: partner invite, KYC decision, shipment assigned, payout released. 2. Register fleet templates: fleet invite, offer received, job assigned, COD due, payout released. 3. Template bodies must match the Meta-approved text exactly or the send is refused locally.	✓ A send with a drifted body fails locally with a clear error rather than at Meta. ✓ Every notification has a fallback channel recorded in <code>notification_templates</code>.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P5-FE2-01	FE-2	fleet	Control room and offer inbox	1. Build FL-04 with a live offer strip above the active-job board. 2. Build FL-05 with fee, weight, pincode, promised_by and a seconds-remaining ring. 3. Accept and decline are optimistic with a rollback on 409, and decline requires a reason code.	✓ A dispatcher accepts an offer in one click and the card moves to the job board. ✓ Losing a race shows a clear “taken by another fleet” state, not a generic error.
P5-FE2-02	FE-2	fleet	Job board and job detail	1. Build FL-06 with columns by state and drag-onto-rider assignment. 2. Build FL-07 masked by default with an explicit Reveal address button. 3. The reveal button states plainly that the access is logged, before it is pressed.	✓ Assigning a rider at the 8-job cap is refused in the UI with the reason shown. ✓ The reveal button copy names the disclosure register.
P5-FE2-03	FE-2	fleet	Rider directory, vehicles and service areas	1. Build FL-13, FL-14, FL-15 for riders and FL-16 for vehicles with expiry countdowns. 2. Build FL-17 and FL-18 with bulk pincode paste and a per-pincode fee grid. 3. Show a pending-approval banner while an area request is outstanding.	✓ A fleet adds 40 pincodes by paste and sees them pending in one screen. ✓ An insurance expiry inside 30 days shows an amber chip on the vehicle row.
P5-FE1-01	FE-1	admin	Fleet management screens	1. Build AD-25, AD-26 with seven tabs, AD-27 invite and AD-28 KYC review with insurance validation. 2. Build AD-29 service-area requests with a coverage delta preview. 3. Build AD-30 the cross-fleet rider directory with suspension.	✓ An admin approves an area request and the fleet immediately receives offers there. ✓ Suspending a rider reassigns that rider’s live jobs and shows what was moved.
P5-FE1-02	FE-1	admin	Job screens and the exception board	1. Build AD-10, AD-11, AD-12 and AD-14. 2. AD-11 shows wave history per job with assign and rebroadcast inline, no navigation. 3. AD-14 renders pincode coverage with the gaps listed, not just the covered set.	✓ An ops user clears five stuck jobs without leaving AD-11. ✓ AD-14 names at least the top ten uncovered pincodes by demand.
P5-FE1-03	FE-1	kit	Kit v0.5: kanban, countdowns and live updates	1. <Board> with drag-and-drop and keyboard-accessible move actions. 2. <Countdown> driven by a server-supplied deadline, not the client clock, with drift correction. 3. A polling hook with a configurable interval and a page-visibility pause.	✓ A countdown is accurate within two seconds after the tab has been backgrounded for an hour. ✓ The board is fully operable by keyboard with an announced move.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P5-BE2-04	BE-2	fleet	Rider assignment rules	1. Attaching a rider requires ACTIVE status, valid documents and fewer than RIDER_MAX_ACTIVE_JOBS. 2. A heavy-equipment shipment requires a PICKUP or TRUCK vehicle with valid insurance. 3. Assignment is idempotent and recorded on the job with a timestamp.	✓ A rider at eight jobs is refused with 409 RIDER_AT_CAPACITY. ✓ A two-wheeler cannot be attached to a 30 kg shipment.
P5-ALL-05	all	demo	Supplier loop demo	1. Demonstrate accept, pack, shortfall, ready, offer broadcast, accept, rider assignment. 2. Do it on production with the internal ATV-SELF fleet and one real supplier. 3. Record the elapsed wall-clock time for each step as the baseline.	✓ The loop completes with no developer intervention. ✓ A baseline timing table exists for W10 comparison.

11.12 W6 — DELIVERY EXECUTION, PROOF OF DELIVERY AND DISCLOSURE

11 tasks · gate: A rider scans pickup, opens the manifest (logged), delivers with OTP inside the geofence, and the order shows DELIVERED in the app. Failed attempts and RTO both work.

Week 6 is where the programme touches a real person outside a building, and it is the only week that deliberately releases a piece of customer data. A rider needs the delivery address and nothing else — never the

phone number — and that release is job-scoped, window-scoped by MANIFEST_WINDOW_MINUTES, and written to pii_disclosures in the same transaction that serves it. If the row is not written the address is not released. Eleven tasks is the lightest count so far because the surface is small; the care per task is the highest in the programme.

The ordering runs backwards from the proof rather than forwards from the screen. The disclosure register and the manifest endpoint land first, then pickup scan, then the OTP-plus-geofence completion, then the failed-attempt path, then RTO. Building the happy path first is the tempting order and it is wrong, because a delivery that can complete before a failed attempt can be recorded produces a rider app that has no way to say no, and the RTO transitions then get bolted onto a state machine that was designed without them.

The specific risk is the geofence and the clock. POD_GEOFENCE_METRES is a policy number, not a tuning parameter, and a device that reports a stale or coarse location will fail a legitimate delivery at a genuine doorstep; the handling of that case has to be a defined refusal with a reason the rider can read, not a silent failure. Delivery OTPs are stored as bcrypt hashes and never as digits, and they are never logged, so debugging a failed completion means reading the disclosure and attempt records rather than grepping for the code. Friday shows one complete delivery inside the geofence with a matching pii_disclosures row, and one deliberately failed three times so that RTO fires automatically.

WATCH FOR

The PII disclosure register must be populating correctly before this week closes.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P6-BE2-01	BE-2	fleet	Pickup scan and the supplier gate handover	1. Implement POST /fleet/jobs/:id/pickup taking the supplier gate code and asserting the job is ACCEPTED or ARRIVED_PICKUP. 2. Assert the shipment is READY_FOR_PICKUP and transition it to PICKED_UP in the same transaction. 3. Emit an order_events row and recompute the parent order status.	✓ A wrong gate code returns 409 GATE_CODE_MISMATCH and is rate limited to five attempts per job. ✓ The supplier sees the shipment leave their queue within one poll interval.
P6-BE1-01	BE-1	fleet	Job manifest release with mandatory disclosure logging	1. Implement GET /fleet/jobs/:id/manifest returning the full delivery address and nothing else that is PII. 2. Gate on job state ACCEPTED..OUT_FOR_DELIVERY and on MANIFEST_WINDOW_MINUTES since acceptance. 3. Call discloseOrThrow() so the pii_disclosures row is written in the same transaction as the read.	✓ Outside the window the endpoint returns 403 DISCLOSURE_WINDOW_CLOSED and writes no row. ✓ The customer phone is absent from the response body under every code path.
P6-BE2-02	BE-2	fleet	Delivery OTP issue and verify	1. Generate a six digit delivery OTP at OUT_FOR_DELIVERY, store only its bcrypt hash on the job. 2. Send the OTP to the customer over the WhatsApp template, never to the fleet portal. 3. Verify on POST /fleet/jobs/:id/deliver with POD_GEOFENCE_METRES enforced against the pin.	✓ The OTP digits appear in no log line, no response body and no Sentry breadcrumb. ✓ A delivery attempted outside the geofence returns 422 OUT_OF_GEOFENCE with the measured distance.
P6-BE2-03	BE-2	fleet	Proof of delivery record and Cloudinary signing for riders	1. Add POST /fleet/jobs/:id/pod-signature issuing a scoped Cloudinary upload signature for the job folder. 2. Persist pod_records with photo public ids, signature image, receiver name and captured coordinates. 3. Reject a deliver call whose job requires POD when no pod_records row exists.	✓ The Cloudinary secret never leaves the API; the rider uploads directly with a 10 minute signature. ✓ A COD job cannot be delivered without both a POD photo and a collected amount.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P6-BE2-04	BE-2	fleet	Failed attempts, reattempt scheduling and RTO initiation	1. Implement POST /fleet/jobs/:id/attempt-failed with a reason code from a closed enum. 2. Allow two reattempts, then move the job to RTO_INITIATED and the shipment to RTO_IN_TRANSIT. 3. Recompute the order; a single failed shipment must not fail the whole order.	✓ Three failed attempts produce exactly one RTO job and one admin exception board entry. ✓ The 28 case reduction test still passes with a mix of DELIVERED and RTO shipments.
P6-FE2-01	FE-2	fleet-portal	Rider job execution flow, FL-11 through FL-16	1. Build the job detail screen as a step machine: accept, arrive, pickup, out for delivery, deliver. 2. Put the manifest behind an explicit tap that shows a one line notice that the view is logged. 3. Capture POD photos through the signed direct upload with an offline retry queue.	✓ Every state transition is a single primary action; no screen offers two competing next steps. ✓ The manifest notice text is present and the disclosure appears in the admin register after use.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P6-FE2-02	FE-2	fleet-portal	Fleet dispatcher board and rider live status, FL-08 to FL-10	1. Render active jobs grouped by rider with the state, the SLA countdown and the last event time. 2. Add a reassign-rider action gated on f.job.assign and blocked once the job is OUT_FOR_DELIVERY. 3. Surface a blocked banner when the fleet ledger balance is negative.	✓ A dispatcher can find any active job in two clicks from the board. ✓ The blocked banner links to the COD remittance screen with the outstanding amount.
P6-FE1-01	FE-1	admin-portal	Delivery control room, exception board and disclosure register	1. Build AD-18 exception board listing UNASSIGNED, breached SLA and stalled jobs with an action per row. 2. Build AD-20 job detail with the full event timeline, offer history and POD gallery. 3. Build AD-38 disclosure register with filters by fleet, rider, reason and date.	✓ Every exception row has exactly one recommended action and one escape hatch. ✓ The disclosure register is read only; no screen in admin can delete a disclosure row.
P6-FE1-02	FE-1	admin-portal	SLA instrumentation and the delivery dashboard tiles	1. Compute per stage SLA breach counts server side and expose them on the dashboard summary endpoint. 2. Add tiles for unassigned jobs, breached pickups, breached deliveries and open exceptions. 3. Make each tile a saved view link rather than a bespoke screen.	✓ Tile counts and the underlying list always agree because both call the same filter. ✓ The dashboard makes at most four requests on load.
P6-BE1-02	BE-1	platform	Notification fan-out for the delivery lifecycle	1. Map each order_events type to zero or one customer notification template. 2. Send through the job_queue worker so a template failure never blocks a transition. 3. Record webhook_deliveries and template send results for the runbook.	✓ A template outage degrades to in-app timeline only and raises no 5xx. ✓ No notification body contains more than the last four digits of any identifier.
P6-ALL-01	all	quality	End-to-end rehearsal on a seeded environment	1. Seed two suppliers, one third-party fleet, ATV-SELF and one customer order spanning both suppliers. 2. Drive the full path from placement to two DELIVERED shipments with the mobile app as the reader. 3. Record the wall-clock time and every manual intervention needed.	✓ The order reaches DELIVERED with no direct write to orders.status anywhere in the trace. ✓ The rehearsal log names every manual step so W7 can automate or accept each one.

11.13 W7 — SETTLEMENT AND CASH ON DELIVERY

10 tasks · gate: Payout accrual, weekly run assembly, lock, release with UTR capture; COD declaration and reconciliation; a synthetic week reconciled to the paisa.

Week 7 is finance-facing and nothing in it may be approximate. Commission accrues per shipment, a weekly run assembles those accruals, the run locks, money leaves, and a UTR comes back and is captured against the run. In parallel a delivery company declares the cash it collected and finance reconciles that declaration against what the system believes was collected. Both halves are integer paise in BIGINT columns and both write to append-only ledgers, so a mistake is corrected by a compensating row and never by an UPDATE. Ten tasks, and every one of them has a number in it that someone will eventually audit.

The sequence is accrual, then run assembly, then lock, then release, and each stage has to be demonstrable on its own before the next is built. M7 installs the three append-only Postgres rules and does not backfill history, which means the first run covers the first full week after go-live only; that is a deliberate limitation stated in the release note rather than a gap to be quietly filled later. COD_VARIANCE_BLOCK_BPS decides when a variance stops being a rounding artefact and starts being a block on the next remittance, and it is a policy value from AA-08, not a knob for a developer to relax when a test fails.

What goes wrong here is almost always ordering rather than arithmetic. A run that is released before it locks can accrue a new shipment mid-flight, so the run total and the sum of its lines disagree by the amount of one shipment and every subsequent week inherits the discrepancy. Idempotency keys on the release call are what stop a retried request paying twice. Friday shows a synthetic week accrued, assembled, locked, released with a captured UTR and reconciled to the paisa, plus one COD period declared by a fleet and reconciled by finance with the variance computed and displayed.

SCREEN NUMBERS IN P7-FE2-01 AND P7-FE2-02 ARE SUPERSEDED

These two cards were written against an earlier screen inventory. SP-18 is the return detail screen and FL-18 is the service-area request screen in the current inventory, so the ranges "SP-18 to SP-22" and "FL-18 to FL-22" quoted in the task titles are wrong at the lower bound. AA-06 section 6.4 is authoritative for supplier screen numbering and AA-07 section 7.3 is authoritative for fleet screen numbering. Build the settlement, ledger, invoice, earnings and COD screens those two documents name; do not build to the numbers printed in the table below. The acceptance lines on both cards are unaffected.

WATCH FOR

Finance-facing. Nothing here may be approximate.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P7-BE2-01	BE-2	money	Partner payout accrual from delivered shipments	1. On shipment DELIVERED, accrue a partner_ledger credit of goods value less commission from the rate card. 2. Compute commission in integer paise with banker-free floor rounding and store the applied bps on the row. 3. Make the accrual idempotent on shipment id so a replayed event cannot double credit.	✓ Replaying the delivered event a hundred times yields exactly one ledger row. ✓ The applied commission bps is stored on the ledger row, not read live from the rate card.
P7-BE2-02	BE-2	money	Payout run assembly, approval and marking paid	1. Implement payout_runs with states DRAFT, APPROVED, PAID, PARTIALLY_PAID and FAILED. 2. Assemble a run from unsettled ledger rows within a period; freeze the membership on approval. 3. Require two distinct staff for approve and mark-paid, both with fresh TOTP.	✓ A DRAFT run recomputes on refresh; an APPROVED run never changes its membership. ✓ The same staff account cannot both approve and mark paid; the API returns 409 SAME_ACTOR.
P7-BE2-03	BE-2	money	Fleet payout accrual per delivery event	1. Accrue fleet_ledger credits from the service area rate card: base, per kilometre and COD handling fee. 2. Debit failed-attempt penalties and RTO fees as separate signed rows, never as an adjusted credit. 3. Expose GET /fleet/ledger with a running balance computed from the rows, not stored.	✓ The ledger balance recomputed from rows equals the balance shown in both portals. ✓ No UPDATE ever touches a ledger row; the Postgres rule proves it in CI.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P7-BE2-04	BE-2	money	COD remittance lifecycle and auto-block	1. Create a cod_remittances row per fleet per day summing collected COD from pod_records. 2. Move it through DUE, DECLARED, DEPOSITED, RECONCILED or SHORT with an admin confirmation at DEPOSITED. 3. Auto-block a fleet from offers when the outstanding balance is negative or variance exceeds COD_VARIANCE_BLOCK_BPS.	✓ A blocked fleet receives zero offers and the reason is visible in both portals. ✓ A SHORT remittance produces a compensating debit row, never an edit of the original.
P7-BE1-01	BE-1	money	Refund, adjustment and wallet credit controls	1. Route every refund through the two-person control built in W2 with a mandatory reason code. 2. Cap a single agent-initiated adjustment at ADJUSTMENT_SINGLE_CAP_PAISE, above which a manager approves. 3. Write the audit diff and the ledger row in one transaction so neither can exist alone.	✓ A refund above the cap without approval returns 403 APPROVAL_REQUIRED and writes no ledger row. ✓ Wallet cash-out remains absent from the API surface; the contract test asserts no such route exists.
P7-FE1-01	FE-1	admin-portal	Finance screens AD-21 to AD-26	1. Build the payouts queue, the run detail with the frozen membership, and the mark-paid flow with step-up. 2. Build the COD reconciliation board grouped by fleet with the outstanding amount first. 3. Build the ledger explorers for wallet, partner and fleet with export to the async masked exporter.	✓ Every money screen shows amounts formatted from paise with no floating point anywhere in the client. ✓ Bank identifiers appear as last four only, including inside exports.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P7-FE2-01	FE-2	supplier-portal	Supplier earnings and settlement screens SP-18 to SP-22	1. Build the earnings summary, the ledger list and the payout history with the run reference. 2. Show the commission bps that was actually applied per shipment, not the current rate card. 3. Add a downloadable statement for a chosen period through the async export path.	✓ A supplier can reconcile any payout back to the shipments that composed it. ✓ The statement totals equal the sum of the ledger rows for the period, to the paise.
P7-FE2-02	FE-2	fleet-portal	Fleet earnings, COD declaration and payout screens FL-18 to FL-22	1. Build the earnings summary with base, distance and COD handling split out. 2. Build the COD declaration flow with the deposit reference and a receipt photo upload. 3. Surface the block state prominently with the exact amount needed to clear it.	✓ A dispatcher can declare a deposit in under a minute from the blocked banner. ✓ Penalty and RTO debits are itemised, never netted into a single opaque figure.
P7-BE1-02	BE-1	platform	Reconciliation crons and the finance safety net	1. Add a nightly cron asserting sum(ledger) equals sum(payouts) plus sum(outstanding) for every org. 2. Add a cron flagging any COD remittance DUE for more than two days. 3. Fail loudly into the exception board rather than silently correcting.	✓ A deliberately corrupted fixture is caught by the nightly assertion within one run. ✓ No cron mutates money; every cron only reads and raises.
P7-ALL-01	all	quality	Money week freeze and a full settlement rehearsal	1. Run a period end on seeded data: accrue, assemble, approve, mark paid, reconcile COD. 2. Deliberately introduce a short remittance and a failed payout and follow both to resolution. 3. Review every ledger row produced by the rehearsal line by line against the expected arithmetic.	✓ The rehearsal ends with every org balance explainable from its rows alone. ✓ The two deliberate faults were resolved through documented runbooks, not ad hoc SQL.

11.14 W8 — RETURNS, RTO AND THE ADVISOR CONSOLE

10 tasks · gate: The full return and RTO loop with compensating ledger entries, and agronomists working the advisory queue from the admin portal within the business-hours SLA.

Week 8 carries two workstreams that have nothing to do with each other, and the only real advice about it is to keep them apart. The return and RTO loop closes the money story: goods come back, a refund goes out, and the

ledger shows a compensating entry rather than a reversed one. The Advisor console is a different product with a different user — an agronomist claiming an advisory request and recording a callback inside a business-hours SLA — and it happens to be scheduled here because it is the last week with any slack in it.

Because the two streams touch different modules they go on separate branches and stay there. A tangled merge between a ledger change and an advisory queue change is the single most avoidable cost in the back half of the programme, and it is avoidable purely by discipline. Within the returns stream the ordering is: return request with photo evidence, approval, pickup, receipt by the supplier, then refund and the compensating ledger row last, because the refund is the only step that cannot be undone.

The trap in this week is the business-hours SLA. Timestamps are stored in UTC and business hours are computed in Asia/Kolkata, so an SLA computed on a client clock or on a UTC calendar day will be wrong by five and a half hours in a way that looks plausible on a Tuesday afternoon and fails at both ends of the day. The second trap is a refund issued before the supplier acknowledges receipt, which turns a return into a loss. Friday shows a return with photo evidence approved, picked, received and refunded with the compensating entry visible in the ledger, and an agronomist claiming a request from the admin portal and logging a callback within the SLA.

SCREEN NUMBERS IN P8-FE2-01 ARE SUPERSEDED

SP-23 to SP-26 are the four supplier settings screens in the current inventory — bank, KYC, users and profile — not returns and quality. The returns surface the card is actually asking for is the returns list and its detail screen. AA-06 section 6.4 is authoritative for supplier screen numbering and AA-07 section 7.3 for fleet; where a task card in this handbook disagrees with either, the design document wins and the card is wrong. Raise it at the Wednesday sync so the whole team hears the correction rather than re-scoping it in a branch.

WATCH FOR

Two unrelated workstreams; keep them on separate branches to avoid a tangled merge.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P8-BE2-01	BE-2	orders	Return request intake with photo evidence	1. Accept a return request per shipment item from the mobile app with up to four signed photo uploads. 2. Validate the request against a return window per category read from the catalog, not hardcoded. 3. Create the return in REQUESTED and put it on the admin returns queue.	✓ A request outside the window returns 422 RETURN_WINDOW_CLOSED with the window in the payload. ✓ The mobile contract for return submission is unchanged from the pre-portal API.
P8-BE1-01	BE-1	orders	Return adjudication with supplier input	1. Give the supplier a comment-and-recommend action; keep the decision with admin staff. 2. Implement approve, reject and partial-approve with a mandatory reason and the audit diff. 3. On approval create a reverse pickup job for the fleet that delivered the shipment where possible.	✓ A supplier recommendation never changes the return state by itself. ✓ A rejected return states the reason to the customer in the notification template.
P8-BE2-02	BE-2	fleet	Reverse pickup jobs and RTO closure	1. Reuse delivery jobs with a direction column rather than creating a second job table. 2. Require POD on reverse pickup: photo plus receiver name at the supplier end. 3. Close the shipment to RETURNED and trigger the refund path once the supplier confirms receipt.	✓ Offer allocation, expiry sweep and fairness ordering all work unchanged for reverse jobs. ✓ A returned shipment reduces the order to a coherent status in every mixed case.
P8-BE2-03	BE-2	money	Refund arithmetic for returns and clawbacks	1. Compute the customer refund to wallet or source per the original payment method. 2. Claw back the partner accrual and the fleet delivery fee as signed compensating rows. 3. Never net a clawback against a future accrual; keep both rows visible.	✓ A full return leaves the three ledgers summing to zero net for that shipment plus retained fees. ✓ Retained fees are named and configurable, not implicit in the arithmetic.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P8-BE1-02	BE-1	advisor	Advisor console backend and the business-hours SLA	1. Compute <code>sla_due_at</code> once at insert using 08:00 to 20:00 IST, seven days a week. 2. Implement claim, add-note, mark-contacted and resolve with the agronomist permission set. 3. Expose a breach list and a per-agronomist load endpoint for the dashboard.	✓ A request created at 19:30 IST is due at 09:30 the next day, and the test asserts it. ✓ A claimed request is invisible to other agronomists until released or timed out.
P8-FE1-01	FE-1	admin-portal	Returns screens AD-14 to AD-17 and Advisor console AD-31 to AD-33	1. Build the returns queue, the return detail with the evidence gallery and the adjudication action. 2. Build the advisory queue sorted by time-to-breach with the claim action as the primary control. 3. Build the advisory detail with the media viewer, the note trail and the resolve flow.	✓ The advisory queue is usable on a laptop at 1366 pixels wide without horizontal scroll. ✓ The customer phone is last four only; contact is a WhatsApp template send, never a tel: link.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P8-FE2-01	FE-2	supplier-portal	Supplier returns and quality screens SP-23 to SP-26	1. Build the returns list with the recommend-and-comment action and no decision control. 2. Build the reverse-pickup expectations view so a supplier knows what is coming back. 3. Build the quality scorecard showing cancellation, breach and return rates with the metric definitions inline.	✓ A supplier cannot see any control that would change a return state. ✓ Every scorecard number links to the list of rows that produced it.
P8-FE1-02	FE-1	admin-portal	Customer screens AD-27 to AD-30 under strict masking	1. Build customer search over the HMAC index so an exact phone or email lookup works without plaintext. 2. Build the customer detail with orders, wallet, advisory history and addresses masked to last four. 3. Add the account action set: block, unblock, force logout, each with reason and audit diff.	✓ Search by partial phone is impossible by design and the empty state explains why. ✓ No screen in the customer area exposes a reveal, copy-full or export-row affordance.
P8-BE1-03	BE-1	platform	Saved views, exports and notification templates completed	1. Finish the async export worker with <code>EXPORT_ROW_CAP</code> enforcement and a signed one-hour download link. 2. Complete the <code>notification_templates</code> registry with a preview endpoint that renders with fake data. 3. Add a template lint asserting no template can interpolate an unmasked identifier.	✓ An over-cap export fails at request time with a clear message rather than half completing. ✓ The template lint fails CI on a deliberately unsafe template.
P8-ALL-01	all	quality	Returns and advisory rehearsal plus the mixed-status matrix	1. Drive a two-supplier order to one DELIVERED and one RETURNED shipment and check the projection. 2. Raise an advisory request at a boundary hour and confirm the SLA due time and the breach alert. 3. Extend the reduction test suite to cover every return and RTO combination.	✓ The mixed-status matrix passes with no manual status writes anywhere. ✓ The advisory breach alert fired to the right channel with no PII in the message.

11.15 W9 — HARDENING, CONTRACT SWEEP AND THE FREE-TIER AUDIT

10 tasks · gate: Twelve CI gates green on five repos, the PII sweep clean, rate limits and CSP in place, eight free-tier thresholds configured and proven to fire.

Week 9 is not polish and it is not a buffer. Four of the twelve CI gates cannot pass until the system they check exists, so they have been advisory or partially satisfied since W1 and this is the week they become true statements: the contract sweep across every non-public route, the permission sweep across every admin route, the CSP assertion on all three deployed portals, and the full M1 to M8 rehearsal against a restored snapshot. A gate that has been failing quietly for six weeks is a gate nobody trusts, which is why they are made blocking here rather than at the end of W10.

The free-tier audit runs in the same week for a reason that is about the budget rather than the code. Eight gauges sit under a Rs 0 recurring commitment, and a threshold that has never fired is a threshold that has never been tested. Each one is proven to fire by pushing it artificially, not by reading a dashboard and agreeing that the number looks low. The paid gaps and their manual workarounds are already named in AA-10; this week only checks that the alarm sounds before the free tier is exhausted rather than after.

The likeliest way to lose this week is to treat the PII sweep as a formality. It is a gate condition, and by W9 there are three portals, forty-one admin screens, twenty-six supplier screens and twenty-four fleet screens in which somebody may have rendered more than four characters of a masked field in a tooltip, a CSV header, a Sentry breadcrumb or an error message. The grep catches ten consecutive digits; it does not catch a masked field concatenated into a log line, and reading the diff is the only control for that. Friday shows twelve gates green on five repositories, a clean PII sweep, rate limits and CSP live on the deployed hosts, and all eight thresholds demonstrated firing.

WATCH FOR

Hardening is not optional polish; the PII sweep is a gate condition.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P9-BE1-01	BE-1	security	PII sweep against the whole surface	1. Run the ten-consecutive-digit grep over every response fixture captured in the contract tests. 2. Assert the Prisma masking extension is unconditional by removing the mask and watching CI fail. 3. Review every pii_disclosures reason code against the five permitted reasons.	✓ The sweep produces zero findings across admin, partner and fleet fixtures. ✓ Removing the masking extension fails at least four distinct CI gates, not one.
P9-BE1-02	BE-1	security	Authorisation matrix test: every endpoint against every role	1. Generate the cross product of endpoints and the seventeen roles across three namespaces. 2. Assert 403 for every unpermitted pair and 200 or a domain error for every permitted pair. 3. Assert cross-tenant access returns 404, never 403, so tenancy is not enumerable.	✓ The matrix is generated from the permission registry, so a new endpoint without a permission fails the build. ✓ A token from one namespace on another namespace returns 401 ACTOR_MISMATCH in every case.
P9-BE2-01	BE-2	quality	Idempotency, concurrency and race hammering	1. Fire concurrent accepts on one offer from three fleets and assert exactly one winner. 2. Replay every idempotent endpoint with the same key under concurrency and assert one effect. 3. Run the expiry sweep concurrently with an accept and assert no job is both expired and accepted.	✓ No test run produces two accepted jobs for one shipment across a thousand iterations. ✓ Every money-moving endpoint rejects a reused key with a different body as 409 IDEMPOTENCY_MISMATCH.
P9-FE1-01	FE-1	admin-portal	Accessibility, keyboard and empty-state pass on admin	1. Run axe over every route and fix every serious and critical finding. 2. Verify keyboard-only completion of the ten most used flows including the step-up modal. 3. Give every list a purposeful empty state that names the next action.	✓ Zero serious or critical axe findings remain and the check runs in CI. ✓ Focus is never trapped and never lost after a modal closes.
P9-FE2-01	FE-2	portals	Accessibility and resilience pass on supplier and fleet portals	1. Repeat the axe and keyboard pass on both portals with the same bar as admin. 2. Test the fleet portal on a 3G-throttled mid-range Android browser and fix the worst three costs. 3. Verify every offline retry queue drains correctly after a network drop mid-upload.	✓ The fleet job screen is interactive within four seconds on throttled 3G. ✓ A mid-upload network drop loses no POD photo and produces no duplicate.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P9-BE1-03	BE-1	platform	Free-tier audit and the degradation drills	1. Measure real usage against every threshold in the budget table and record the headroom. 2. Drill two degradations: Cloudinary credit exhaustion and the WhatsApp template pause. 3. Confirm every alert threshold actually fires by forcing a synthetic breach.	✓ The audit records measured headroom per service, not estimates. ✓ Each degradation has a rehearsed switch that an ops person can flip without a deploy.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P9-FE1-02	FE-1	kit	Kit v1.0, contract freeze and the visual regression baseline	1. Cut kit v1.0 with a written changelog and a deprecation policy for breaking changes. 2. Capture the visual regression baseline for every kit primitive and every masked component. 3. Pin all three portals to the exact version and prove the pin with the CI gate.	✓ No portal resolves a range for the kit; all three lockfiles show the same exact version. ✓ A deliberate colour change in a primitive fails the visual regression gate.
P9-BE2-02	BE-2	quality	Load shape and cold-start behaviour on Cloud Run	1. Measure p95 for the twelve hottest endpoints at pilot-scale concurrency. 2. Measure the cold-start penalty and decide the minimum-instance question explicitly against I-14. 3. Add indexes only where the measurement justifies them and record each decision.	✓ Every hot endpoint is under the agreed p95 or has a written exception with a reason. ✓ The minimum-instance decision is recorded as a ₹0 choice with its stated cost in latency.
P9-ALL-01	all	quality	Runbook rehearsal by someone who did not write the runbook	1. Assign each of RB-01 to RB-11 to an engineer who did not author it. 2. Execute each runbook verbatim on a restored branch and record every point of confusion. 3. Rewrite every step that needed interpretation.	✓ All eleven runbooks were executed end to end by a non-author. ✓ No runbook step requires knowledge that is not in the runbook.
P9-ALL-02	all	quality	Definition-of-done audit against D-01 to D-20	1. Walk every definition-of-done item and mark it met, partially met or not met with evidence. 2. Convert every partially-met item into either a W10 task or an accepted, written exception. 3. Publish the audit as the go or no-go input for the pilot.	✓ No item is marked met without a link to the test, screenshot or log that proves it. ✓ Every exception names who accepted it and what the follow-up is.

11.16 W10 — PILOT, RUNBOOKS AND CUTOVER

9 tasks · gate: Two real product companies and one real delivery company transact end to end. Eleven runbooks written and rehearsed. Go or no-go decided on evidence.

Week 10 has no new features in it. Nine tasks, the fewest of any week, and the deliverable is evidence rather than software: two real product companies and one real delivery company transacting end to end, eleven runbooks written and each rehearsed once by somebody other than its author, and a go or no-go call made on what actually happened rather than on how close it felt. Anything that arrives as a feature request this week is a Phase 2 line item, and the correct answer is to write it down and not build it.

The bug budget is the mechanism that keeps this honest. Fixes are triaged against the twenty definition-of-done demonstrations: a defect that stops a D-number being demonstrated is fixed, and a defect that is merely ugly is logged. Runbook rehearsal is deliberately assigned away from the author, because a runbook you wrote is a runbook you can follow from memory, and the point of the rehearsal is to find the step that was obvious to the author and is missing from the page.

What tends to go wrong in the final week is not technical. Real pilot partners generate data the team has never seen — a company name with a character the KYC form rejects, a service area drawn around a district that no rate card covers, a COD declaration submitted on a Sunday — and each one looks like a bug and is often a missing decision. Reserve capacity for that, and exercise the cut list from AA-10 section 10.6 in ID order the moment the week is behind rather than at the end of it. Friday is the go or no-go: the twenty demonstrations, the runbook rehearsal log, the monthly bill and the eight gauges, in that order.

WATCH FOR

No new features. Bug budget only. The cut list is exercised here if needed.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P10-ALL-01	all	pilot	Pilot cohort selection and the written pilot contract	1. Choose three suppliers and two delivery companies plus ATV-SELF as the pilot cohort. 2. Agree in writing the pilot scope, the support hours, the escalation path and the exit criteria. 3. Set the pilot to one district so serviceability is small enough to reason about.	✓ Every pilot org has signed the scope note and knows who to call. ✓ The pilot geography is a single district with a named list of pincodes.
P10-BE1-01	BE-1	ops	Onboarding the real orgs and seeding their data	1. Run each partner and fleet through the real KYC review flow, not a bypass. 2. Configure rate cards, service areas and category licences from the signed paperwork. 3. Create the staff accounts with least privilege and enforce TOTP enrolment on first login.	✓ No pilot org was created by direct SQL; every one went through the reviewed flow. ✓ Every staff account has TOTP enrolled and no account holds SUPER_ADMIN except the two named owners.
P10-FE1-01	FE-1	ops	Portal training material and in-product guidance	1. Write a one-page quick start per portal, in English and Hindi, screenshot led. 2. Add first-run guidance to the three screens that testing showed people got stuck on. 3. Record a five-minute screen capture per portal for the pilot orgs.	✓ A pilot user completes their core loop from the quick start with no phone support. ✓ The guidance is data driven: each of the three screens was chosen from rehearsal notes.
P10-BE2-01	BE-2	cutover	Cutover plan, legacy retirement and the observation window	1. Publish the ordered cutover checklist with the abort condition at each step. 2. Keep orders.legacy_status populated for two releases and compare it daily against the projection. 3. Retire the old console only after seven consecutive clean comparison days.	✓ The legacy comparison ran daily with zero mismatches before retirement. ✓ Every checklist step has an abort condition and a named decision maker.
P10-ALL-02	all	ops	On-call rotation, alert routing and the first-week watch	1. Define a light on-call rotation with the alert routing per severity and a response expectation. 2. Tune alerts during the first pilot week until the noise floor is near zero. 3. Hold a fifteen-minute daily review for the first five pilot days.	✓ Every alert that fired in week one was either actionable or removed. ✓ The rotation is written down with names, hours and the escalation path.
P10-FE2-01	FE-2	portals	Pilot feedback triage and the fast-fix budget	1. Reserve two days per portal for fixes arising from pilot feedback. 2. Triage feedback into blocking, friction and wish, and only fix the first two in the window. 3. Record every wish item into the post-pilot backlog with the requesting org named.	✓ No blocking pilot issue stayed open for more than one working day. ✓ The wish backlog is written down rather than carried in anyone memory.
ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P10-BE1-02	BE-1	ops	Backup, restore and the disaster drill	1. Verify the Neon branch and export schedule and record the actual recovery point objective. 2. Restore a full environment from backup into a scratch project and run the smoke suite against it. 3. Time the restore and write the number into RB-01.	✓ A full restore was completed and timed, and the smoke suite passed against the restored copy. ✓ The recovery point and recovery time objectives are stated numbers, not aspirations.
P10-ALL-03	all	handover	Documentation reconciliation against the shipped system	1. Diff every document in this set against what actually shipped and correct the fact base. 2. Regenerate all thirteen documents from the corrected fact base rather than editing prose. 3. Record every accepted deviation from the plan with the reason.	✓ The regenerated set matches the shipped system, including counts and identifiers. ✓ Every deviation is written down; none is discovered later by a new engineer.

ID	DEV	AREA	TASK	STEPS	ACCEPTANCE
P10-ALL-04	all	handover	Post-pilot review and the Phase 2 shortlist	1. Review the pilot against the exit criteria and decide widen, hold or roll back. 2. Rank the post-pilot backlog by pilot-evidenced pain, not by internal preference. 3. Name the top five Phase 2 items with a rough size each.	✓ The widen decision is made against written criteria, not sentiment. ✓ The Phase 2 shortlist cites the pilot evidence behind each item.

11.17 WHAT TO DO WHEN A WEEK IS LATE

One escalation path, 8 cuts in ID order, and 8 things that are never traded

Some week in this programme will be late. The plan does not depend on that not happening; it depends on it being handled the same way every time, because an improvised response costs more than the slip. There is exactly one path. You notice on Tuesday or Wednesday that a task will not be demonstrable on Friday, you raise it at the Wednesday 18:00 blocker sync in plain terms — the task ID, what is blocking it, and your honest estimate of the remaining work — and the team decides there and then whether the week can still pass. Nobody is judged for raising it on Wednesday. Discovering it at 16:00 on Friday is a different matter, because by then no decision is available.

On Friday the gate is binary. It passes or it fails; there is no partial credit and no "done except for". A failed gate does two things immediately. The unfinished work moves into the following week, which means the following week is now over-committed by exactly that amount. And the next cut is taken from the list below in ID order, C-1 first, to create the capacity that the moved work has just consumed. Taking cuts in ID order rather than by convenience is the point of the list: the order was decided in AA-10 section 10.6 while the team was calm, and re-deciding it under pressure produces a choice that protects whoever is loudest in the room.

ID	CAPABILITY CUT	WHAT SHIPS INSTEAD	DAYS	WHAT IT COSTS
C-1	Fleet SSE offer stream	Fall back to 10 s polling everywhere.	2 dev-days	Riders will not notice; dispatch sees a few seconds of extra latency.
C-2	Saved views and CSV export	Ship fixed default filters only.	3 dev-days	Ops will ask for it within a month. It is a strong Phase 2 candidate.
C-3	Self-billed commission invoice PDF	Supplier downloads a CSV of the run instead.	2 dev-days	Accounting can generate the invoice manually for two pilot suppliers.
C-4	Custom roles	Ship the nine system roles only; no role editor screen.	4 dev-days	Nine roles cover every seat we can name today.
C-5	Rider mobile scan camera	Manual shipment-code entry only.	2 dev-days	Slower at the gate. Acceptable for a pilot with two suppliers.
C-6	POD photo capture	OTP plus geofence only; no photo.	1 dev-day	Weakens dispute evidence. Take this cut only for equipment-free, low-value routes.
C-7	Vehicle registry	Riders exist; vehicles do not. Heavy equipment goes to ATV-SELF.	3 dev-days	Blocks third-party heavy-equipment delivery. Acceptable at pilot scale.
C-8	Service-area approval workflow	Admin edits fleet areas directly.	3 dev-days	Fine with three fleets. Does not scale past ten.

All eight cuts together come to 20 dev-days, which is the whole of the slack this programme has. Note what the ordering encodes. C-1 Fleet SSE offer stream and C-3 Self-billed commission invoice PDF are cheap and nearly invisible: polling replaces the stream and a CSV replaces the PDF, and the pilot will not notice either. C-5 Rider mobile scan camera and C-6 POD photo capture are cheap in days and expensive in evidence, so C-6 in particular is only taken on low-value routes where a dispute is unlikely. C-4 Custom roles and C-7 Vehicle registry are the two that quietly reduce what the product is, and by the time the list has run that far the honest conversation is about scope rather than about days. C-2 Saved views and CSV export and C-8 Service-area approval workflow both survive a pilot and both come back within a month of real use.

Nothing on the following list is ever traded, in any week, for any reason, including a failed gate in W10 with a pilot partner waiting. These eight items are not features; they are the properties that make the system auditable, authorised and honest about money. A programme that ships without them has not shipped early, it has shipped something that cannot be operated. If the only way to pass a gate is to trade one of them, the correct outcome is a failed gate and a conversation with the programme owner, and AA-10 section 10.10 covers

the three conditions under which stopping is the right call.

- The 88-permission matrix with DENY-wins. A portal without authorisation is not shippable.
- The last-four-only masking extension and the pii_disclosures register.
- assertTransition on shipments and delivery jobs. Without it the data becomes unauditible.
- The append-only ledgers and the integer-paise rule.
- Idempotency keys on money and on offer acceptance.
- Audit logging of every admin mutation.
- Mandatory TOTP for staff.
- The M4 backfill rehearsal on a restored snapshot.

THE ESCALATION PATH IN ONE LINE

Notice it early, raise it at Wednesday 18:00 with the task ID, let the Friday gate pass or fail honestly, and if it fails take the next cut in ID order and never one from the list above. Everything else — heroic weekends, quietly reduced acceptance criteria, a demo run against localhost or against seeded fixtures — buys one week and costs three.

One last thing about this handbook. It is the most detailed document in the set and it is also the one most likely to be wrong first, because it commits to an ordering ten weeks in advance. When it disagrees with a design document, the design document wins: AA-00 for architecture, AA-02 for module boundaries and tenancy, AA-03 for the data model, AA-04 for the API contract, AA-05 to AA-07 for screens, AA-08 for permissions and PII, AA-09 for runbooks and AA-10 for the schedule, the cut list and the risks. Keep the acceptance lines, correct the cards, and raise the correction where the whole team can hear it.